Anàlisi d'Apks



1. Introducció **2**
Principis de ciberseguretat* 2
Confidencialitat 2
Integritat 3
Disponibilitat 3
No-repudiament 3
Autenticació 3
2. Pràctica **4**
3. Anàlisi estàtic **10**
Búsqueda d'informació 11
Anàlisi de l'aplicació 18
4. Anàlisi dinàmic **23**
Instal·lació de l'aplicació 23
5. Anàlisi de xarxa **29**
6. Referències **31**

C037 – RA1

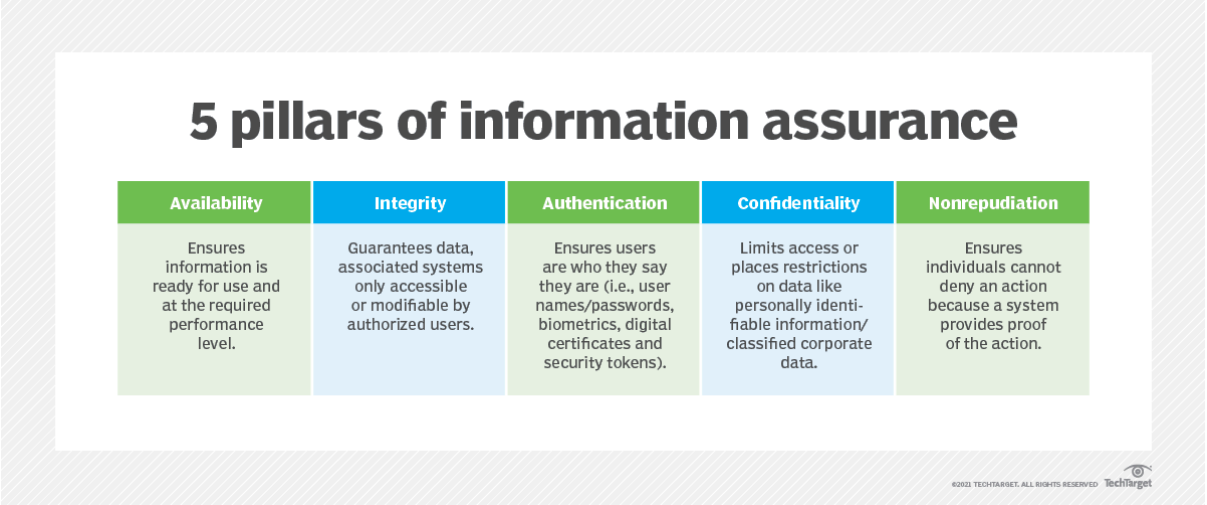
Pràctica 1.3: Análisis apk

1. Introducció

Principis de ciberseguretat*

La ciberseguretat és la pràctica de protegir la informació i els sistemes que emmagatzemen o processen la informació.
Està definida per cinc principis:

- Confidencialitat
- Integritat
- Disponibilitat
- No-repudi
- Autenticació



Confidencialitat

La informació té confidencialitat si només pot ser accedida i llegida per usuaris autoritzats. Els usuaris autoritzats solen incloure la persona que genera la informació i els destinataris previstos de la informació. Violentar la confidencialitat és sovint l'objectiu de molts ciberatacs. Per violar la confidencialitat, els atacants poden interceptar la informació mentre es troba en trànsit (com en una connexió Wi-Fi no segura o a través d'Internet), o poden eludir els controls de seguretat en un sistema per robar la informació mentre es troba en repòs.
La informació comuna objecte d'atacs inclou comunicacions personals (correu electrònic, missatges de text), imatges, secrets comercials, informació de pagaments (números de targeta de crèdit/de dèbit), identificadors personals (números de la Seguretat Social) i informació sensible del govern i l'exèrcit.
La xifra i el control d'accés són mecanismes típics utilitzats per protegir la confidencialitat.

Integritat

La informació té integritat si només pot ser modificada per usuaris autoritzats. L'integritat ha de ser verificable, el que significa que ha de ser fàcil de determinar si la informació ha estat modificada per un tercer no autoritzat.
La integritat pot ser violada mentre la informació es troba en trànsit o en repòs, i aquesta violació pot ser accidental o intencionada. Incidents accidentals inclouen errors en l'entrada de dades, fallida del maquinari i efectes de la radiació solar. Incidents intencionats inclouen modificació no autoritzada d'un fitxer, base de dades o paquet de xarxa.
Freqüentment s'utilitza el hashing criptogràfic per verificar la integritat de la informació.

Disponibilitat

La informació es considera disponible si es pot accedir quan i on sigui necessària. L'accés a la informació també ha de ser oportú i convenient per a l'usuari.
Els atacs contra la disponibilitat estan guanyant popularitat entre els estats-nació i els hacktivistes, ja que tenen un efecte immediat i visible. Els incidents accidentals inclouen la pèrdua de corrent, fallida del maquinari o fallida del programari. Les accions intencionades inclouen els atacs de denegació de servei distribuïts (DDoS) i els atacs de ransomware.
La redundància, les còpies de seguretat de dades i energia, i els llocs de recuperació de desastres són típicament utilitzats per mantenir alts nivells de disponibilitat.

No-repudiament

El no-repudiament vincula una entitat (usuari, programa, etc.) amb les accions realitzades per aquesta entitat. Per exemple, la signatura d'una persona en un contracte legal pot ser utilitzada per demostrar que la persona va acceptar els termes del contracte. És difícil per a la persona que va signar el contracte negar o repudiar posteriorment aquest fet perquè existeix l'evidència de la signatura.
Els mètodes comuns per assegurar el no-repudiament inclouen l'autenticació de l'usuari, les signatures digitals i la registre del sistema.

Autenticació

L'autenticació es refereix a la identificació positiva i la verificació de la identitat d'un usuari. Aquest és un component crític per assegurar que només els usuaris autoritzats puguin accedir o modificar la informació. Els mecanismes d'autenticació són un dels aspectes més objectius dels sistemes d'informació, ja que l'èxit dels altres quatre principis sovint depèn d'això.
Els mecanismes comuns utilitzats per a l'autenticació inclouen noms d'usuari i contrasenyes, targetes d'identificació electròniques i biometria.

2. Pràctica

En aquesta pràctica veurem diferents formes d'analitzar una aplicació de dispositius android (apk). Veurem:

- Com està feta
- Analitzarem el seu codi
- Organització
- Permisos

Realitzarem l'anàlisi en tres fases:

- anàlisis estàtic
- dinàmic
- ús de la xarxa de l'aplicació.

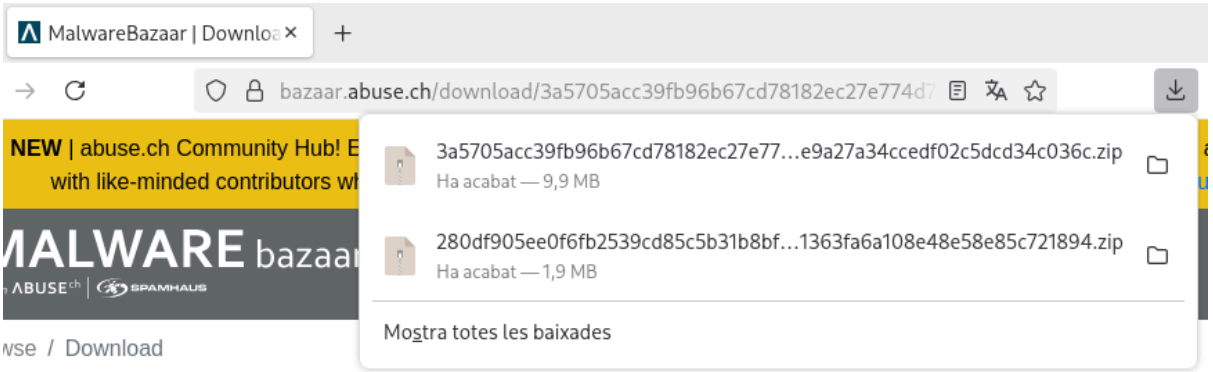
Per avaluar l'aplicació ho farem mitjançant una aplicació no segura. I, per obtenir aquesta aplicació, buscarem una aplicació que sigui sospitosa de contenir malware. Busquedes que ens poden portar a aquestes aplicacions son:

- Free
- Crypto
- Gems
- Premium, plus, … (Qualsevol aplicació que tingui versió premium de pagament (Streaming, jocs, comunicació, …))

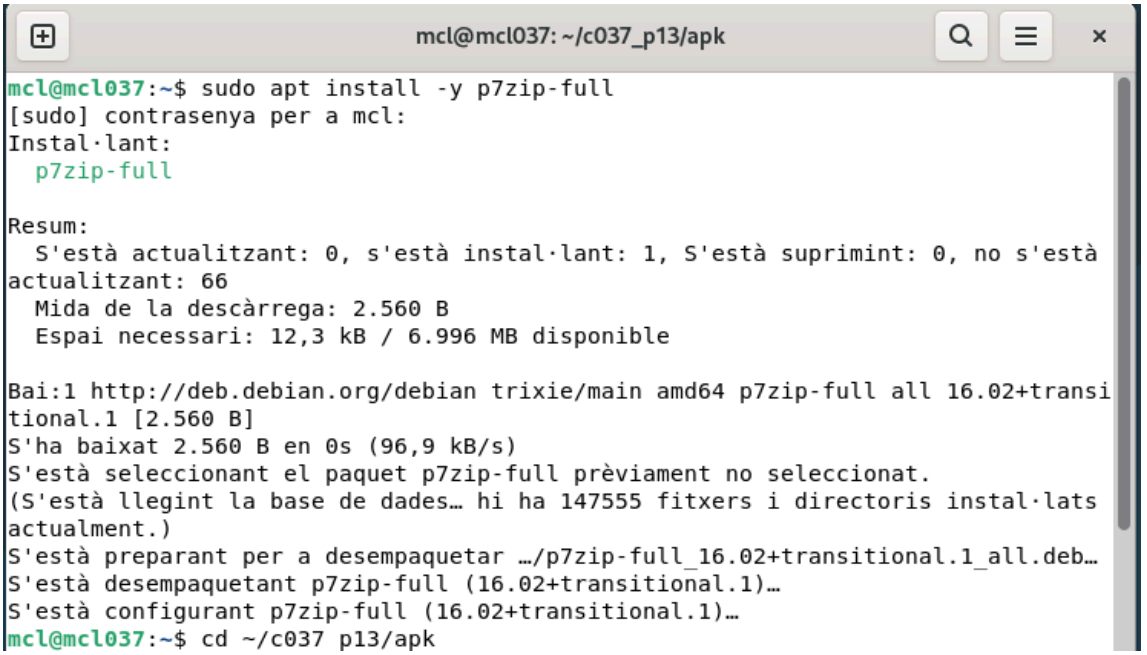
Busca i selecciona 2 aplicacions malicioses. Aquestes seran les que analitzarem.
Puja les aplicacions a la web de virustotal.com i observa:

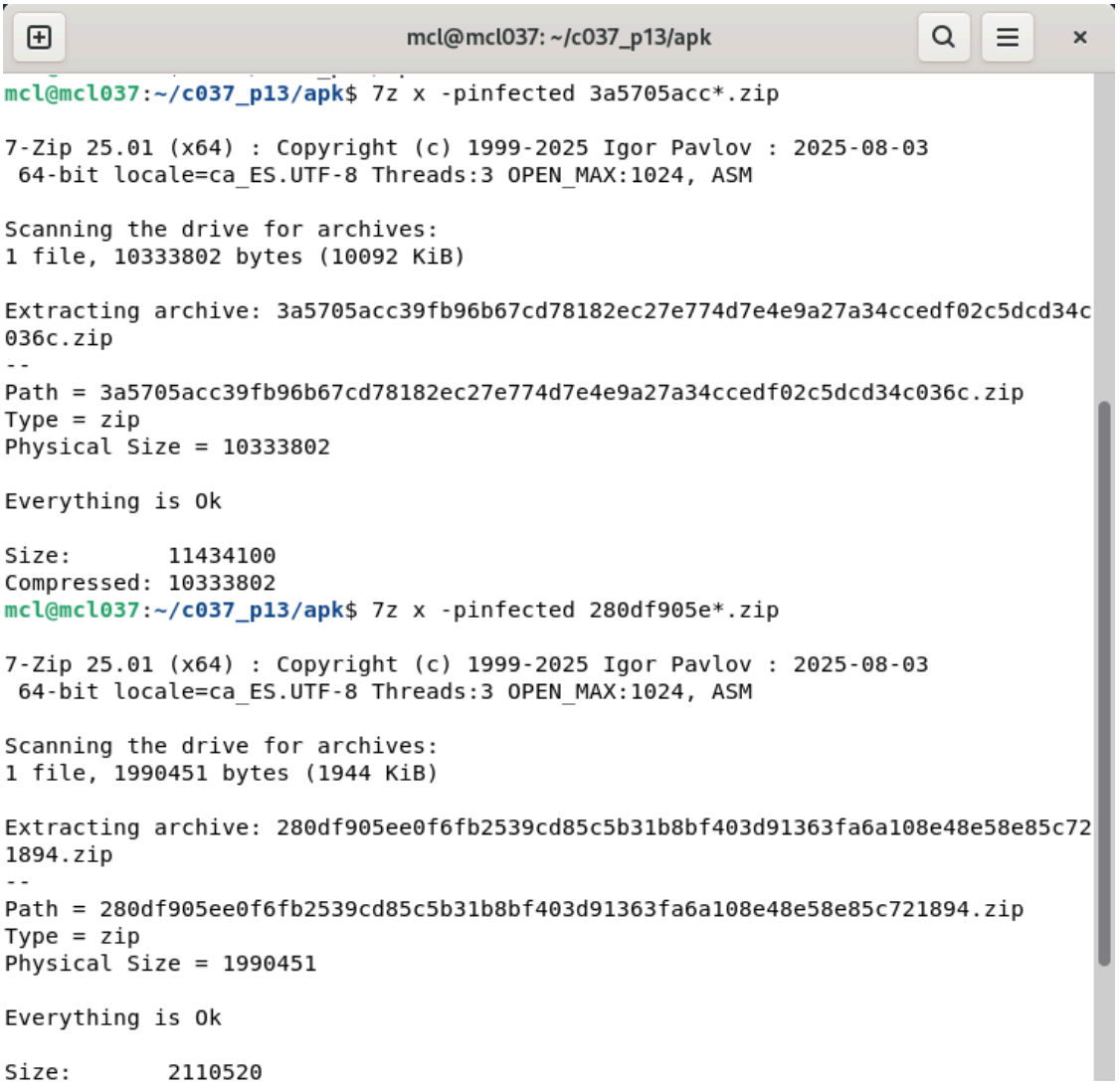
- L'*score* amb el que marca l'aplicació
- Quants *vendors* marquen l'aplicació com a maliciosa.
- L'informe generat a la pestanya de *Details*.

He baixat dues mostres Android catalogades com a malware a MalwareBazaar, una d'una app bancària (ICICI) i una d'una cadena de supermercats (DMart). Les he triat per ser de tipus diferent i perquè pesen poc (9,9 MB i 1,9 MB).

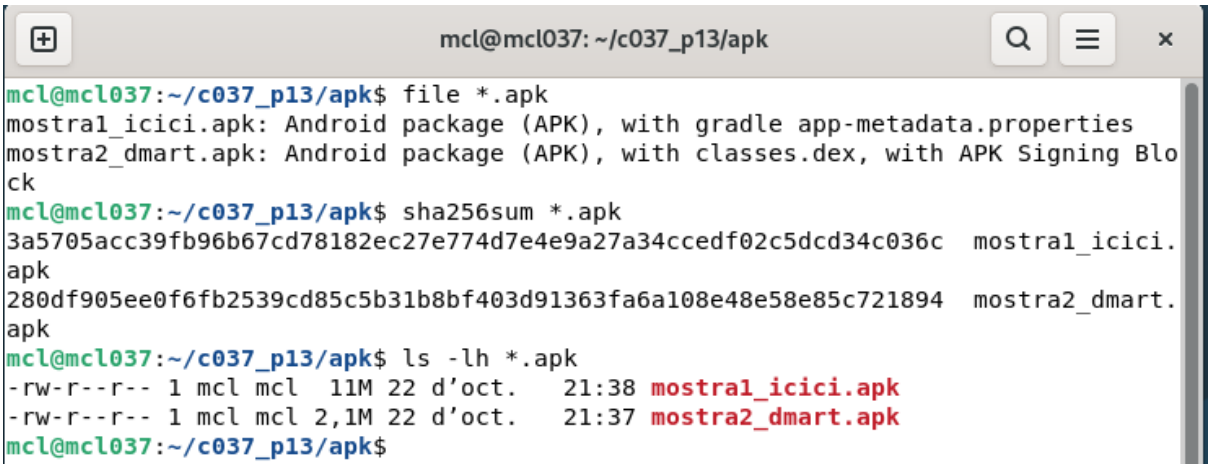


Unzip va fallar perquè els ZIP de MalwareBazaar estan xifrats amb AES, així que he instal·lat p7zip-full i els he extret amb la contrasenya infected. Ha sortit un APK de cada ZIP.

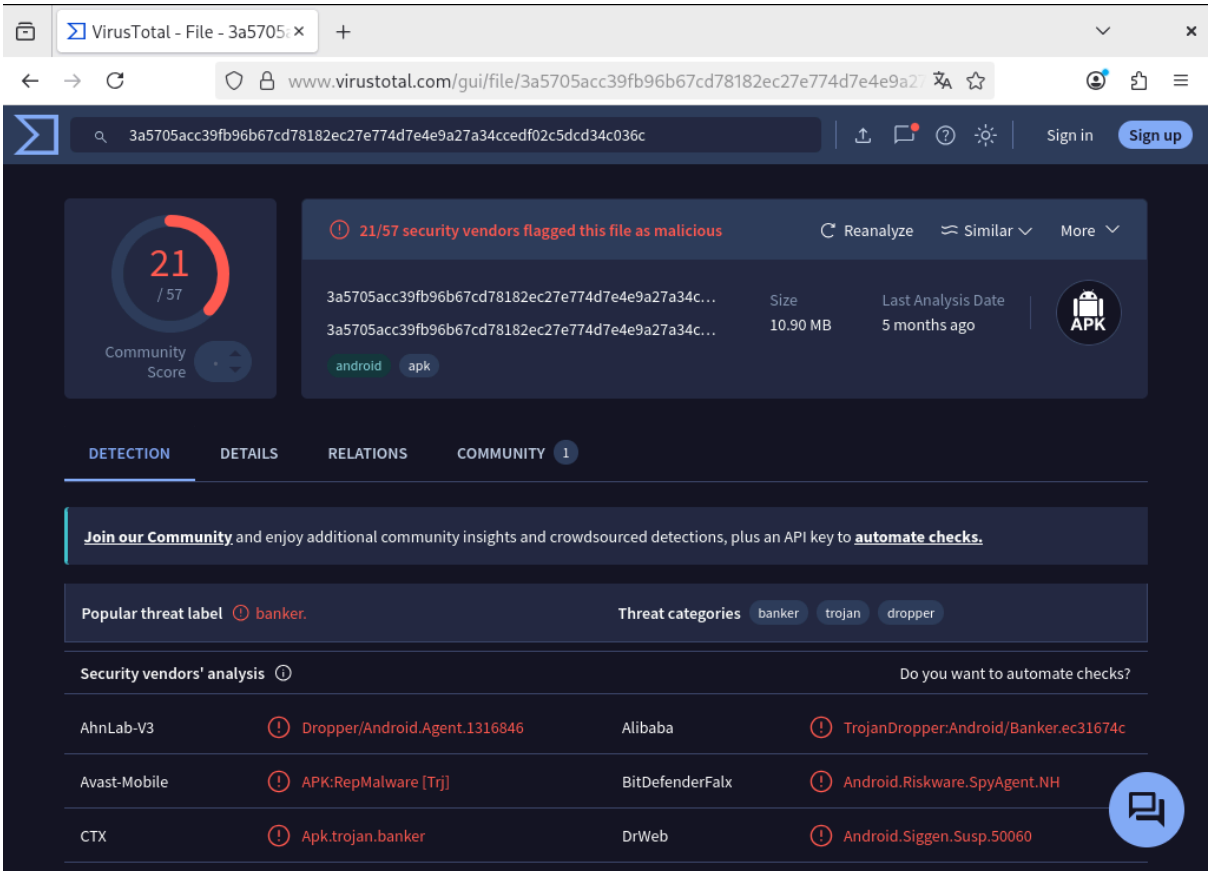




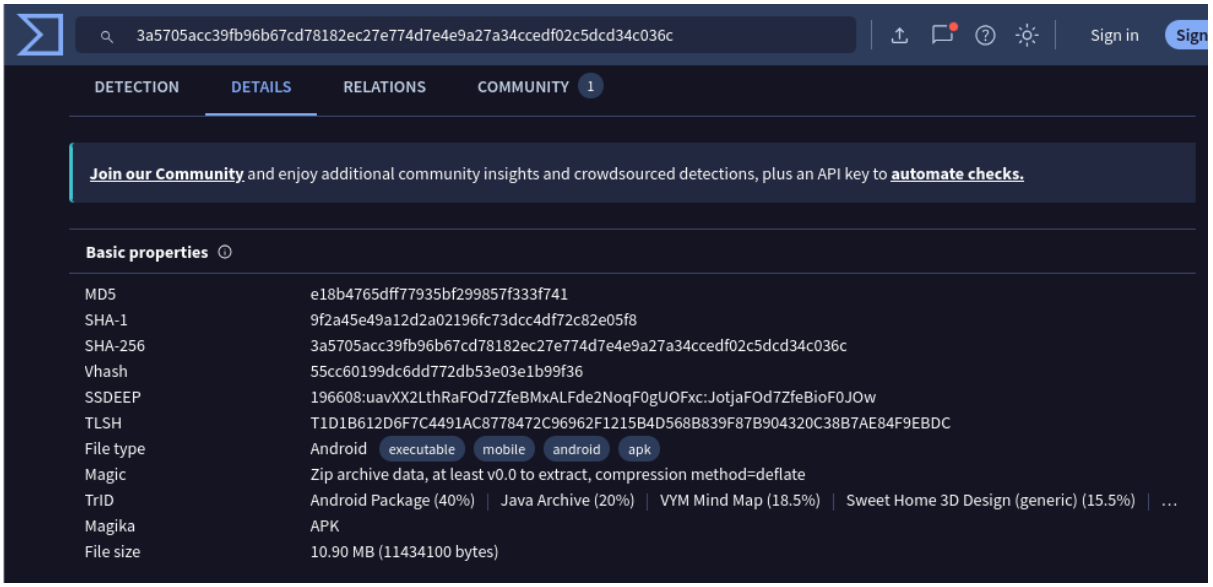
He reanomenat els APK a mostra1_icici.apk i mostra2_dmart.apk. Amb file comprovo que són paquets Android, i amb sha256sum i ls -lh calculo el hash i la mida. Els hashes coincideixen amb els de MalwareBazaar i VirusTotal, així que són les mostres correctes.



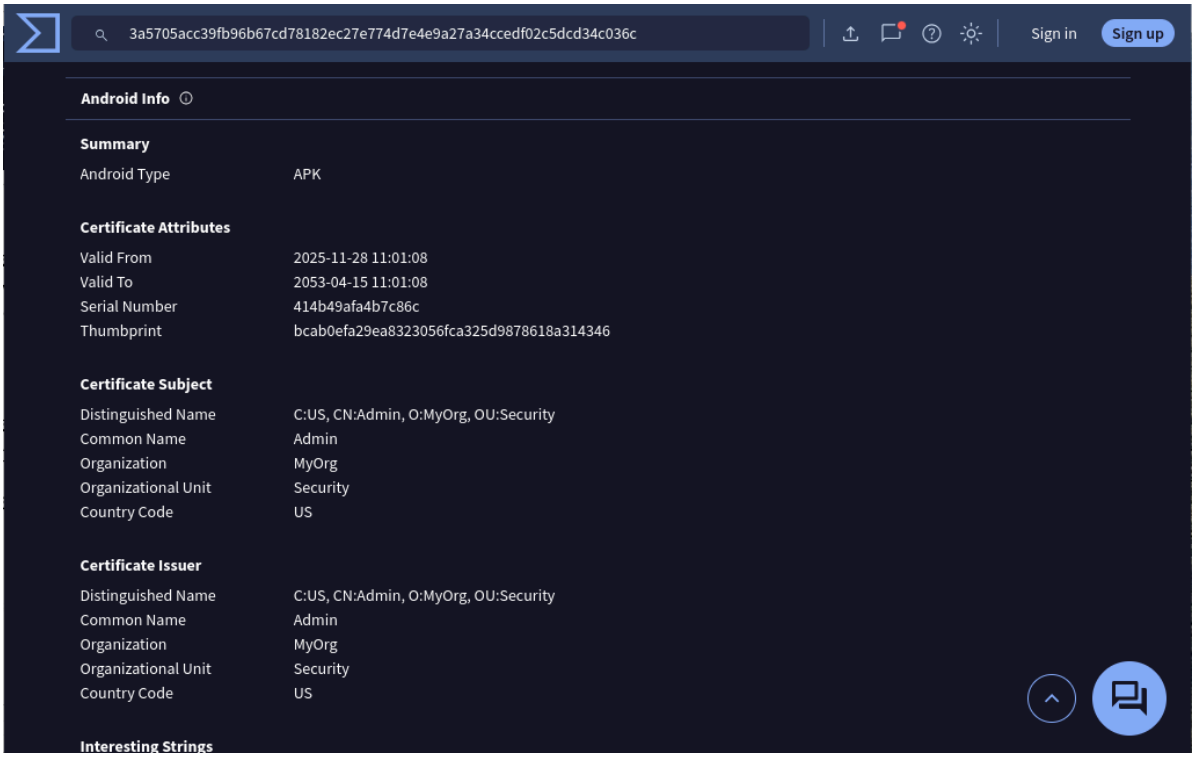
He cercat el SHA256 de la mostra 1 a VirusTotal. 21 de 57 vendors la marquen com a maliciosa, amb l'etiqueta banker i les categories banker, trojan, dropper. Això ja em dona una primera idea del tipus de malware.



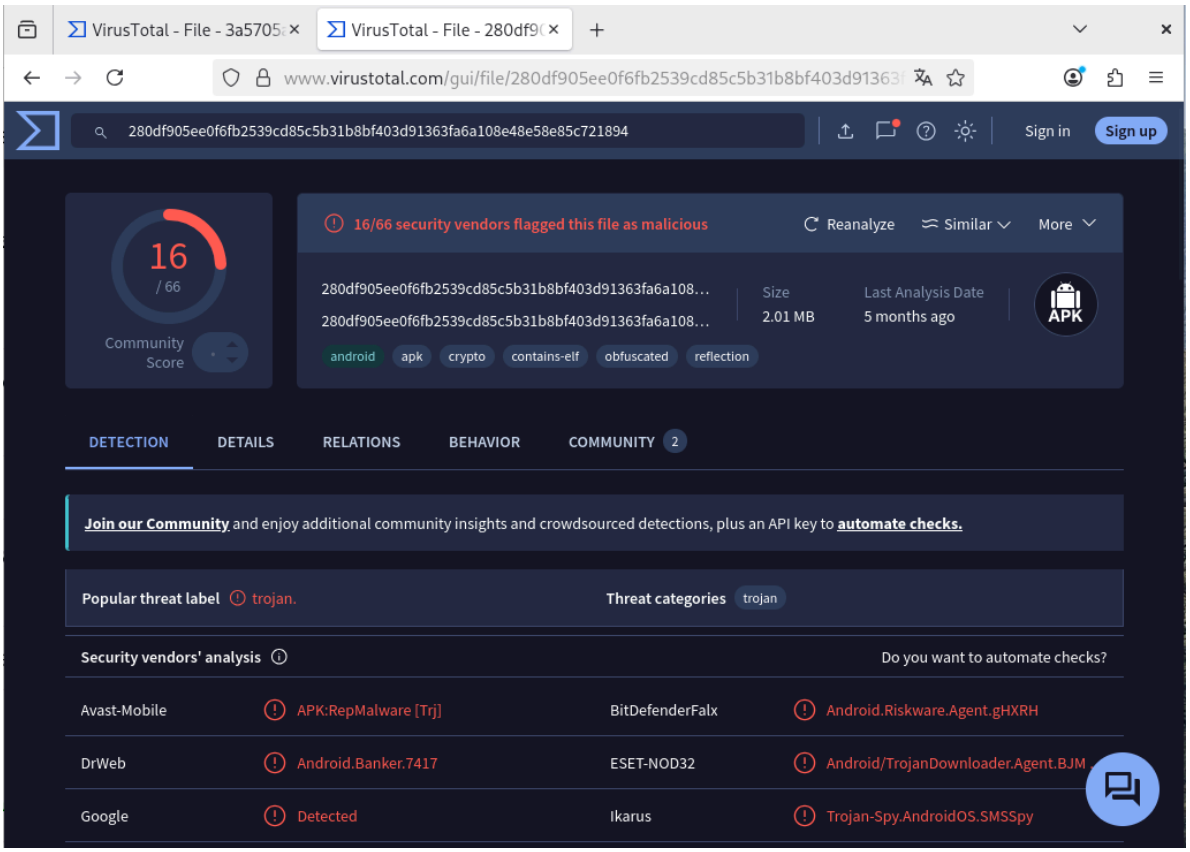
Hi comprovo els hashes (MD5, SHA-1, SHA-256) i la mida. El SHA256 i els 10,90 MB coincideixen amb el fitxer que he calculat a la VM, així que és la mostra correcta.



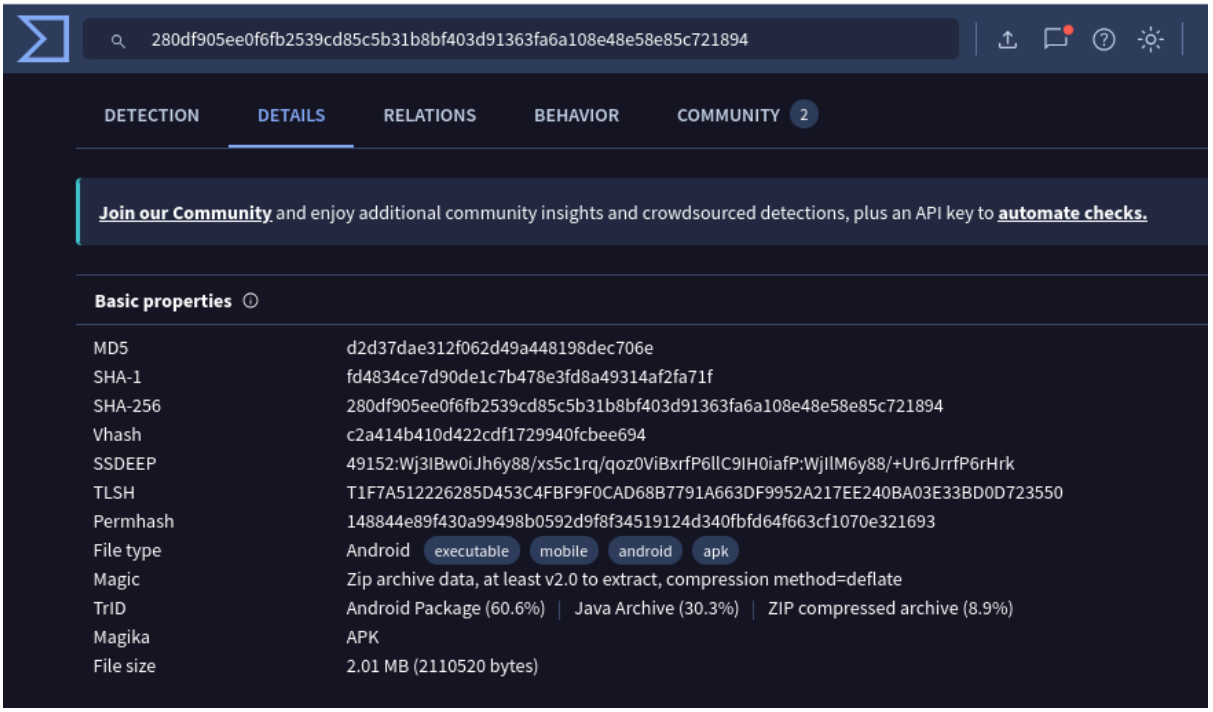
Només hi surt el certificat, sense paquet, SDK ni permisos. El certificat és genèric (CN=Admin, O=MyOrg, OU=Security) i no correspon al d'un banc real. Que VirusTotal no mostri el manifest indica que està manipulat.



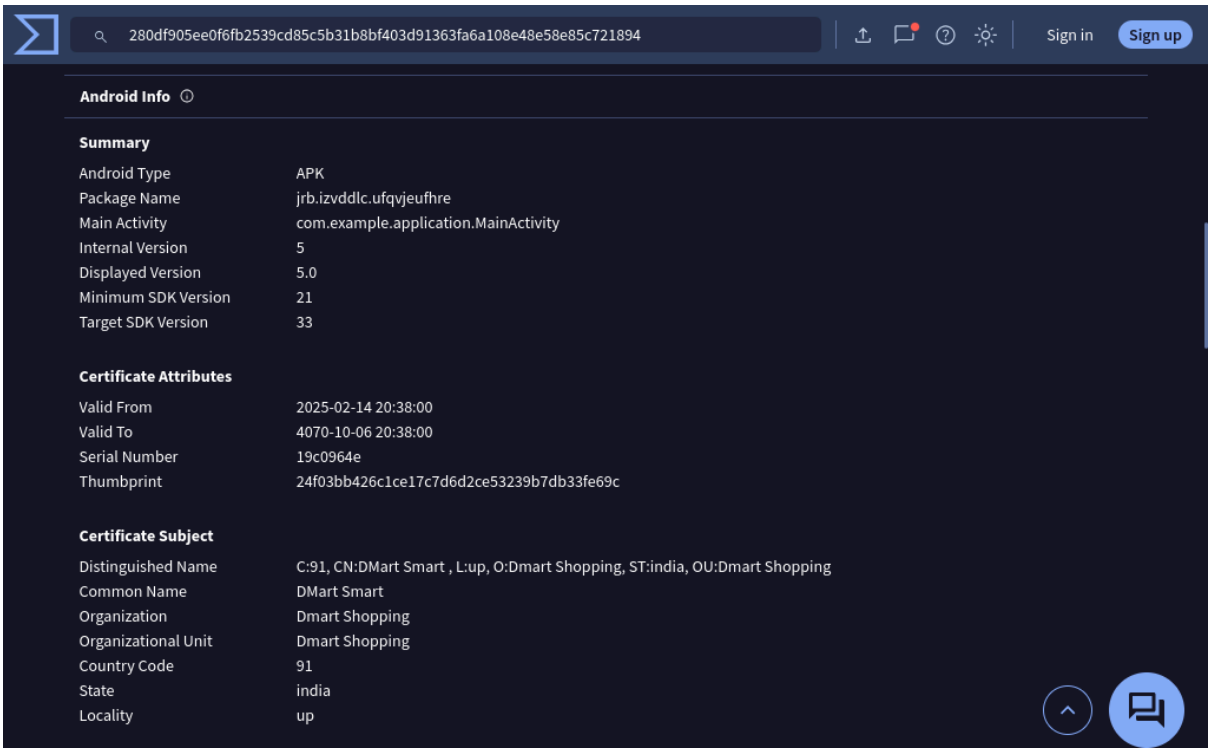
He cercat el SHA256 de la mostra 2. 16 de 66 vendors la marquen com a maliciosa, amb l'etiqueta trojan. Els tags obfuscated, reflection i contains-elf anticipen que el codi està ofuscat i porta llibreries natives.



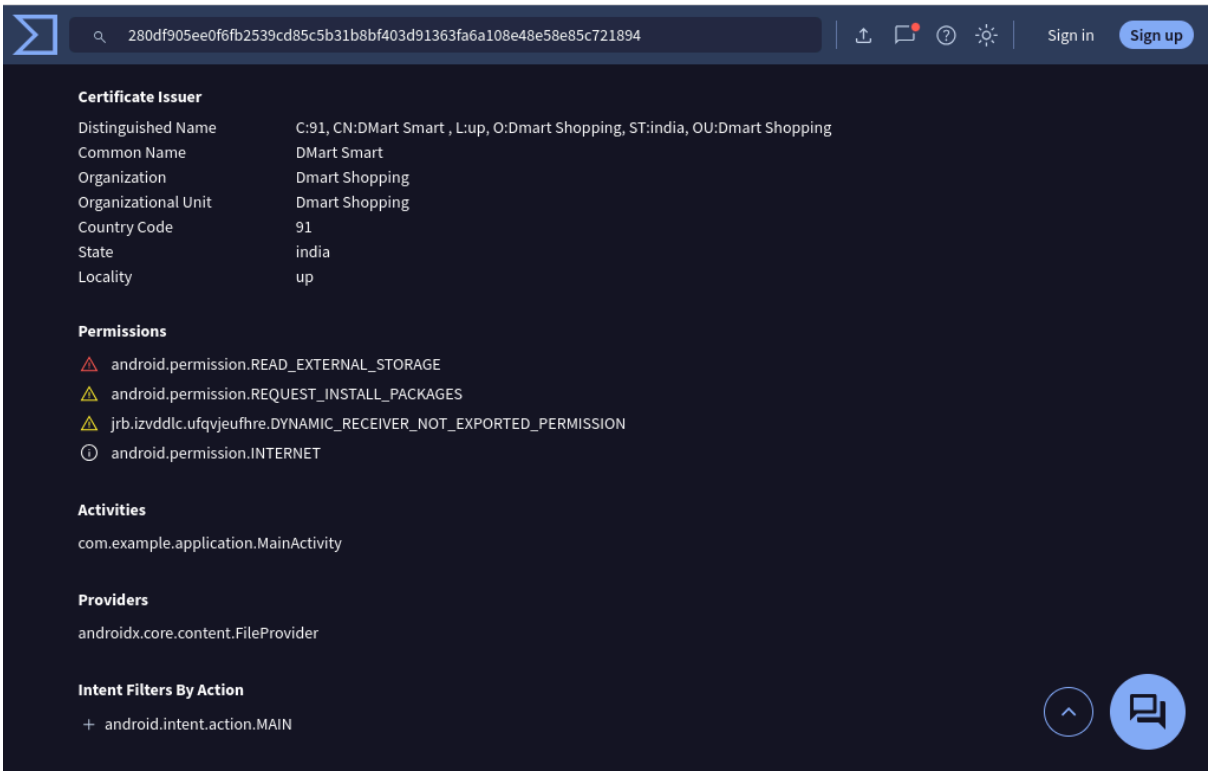
Comprovo hashes i mida. El SHA256 i els 2,01 MB coincideixen amb el fitxer de la VM, així que és la mostra correcta.



Aquí sí que hi ha el manifest. El paquet és jrb.izvddlc.ufqvjeufhre, un nom aleatori, i la main activity és com.example.application.MainActivity. El Min SDK és 21 i el Target SDK 33. El certificat diu ser de “DMart Shopping”, però el nom del paquet no té res a veure amb DMart.



Hi surten els permisos, una activity (MainActivity) i un provider (FileProvider). Destaca REQUEST_INSTALL_PACKAGES, que permet instal·lar altres APK.



3. Anàlisi estàtic

Per aquesta part de l'anàlisi, farem servir les següents eines:

- apktool (Descomprimir l'apk. cli)
- jadx (Eina per descompilar un paquet. cli)
- dex2jar (Eina per descompilar un paquet. gui)

He instal·lat apktool, jadx, adb i dex2jar a la VM i he comprovat que funcionen amb la versió de cadascun.

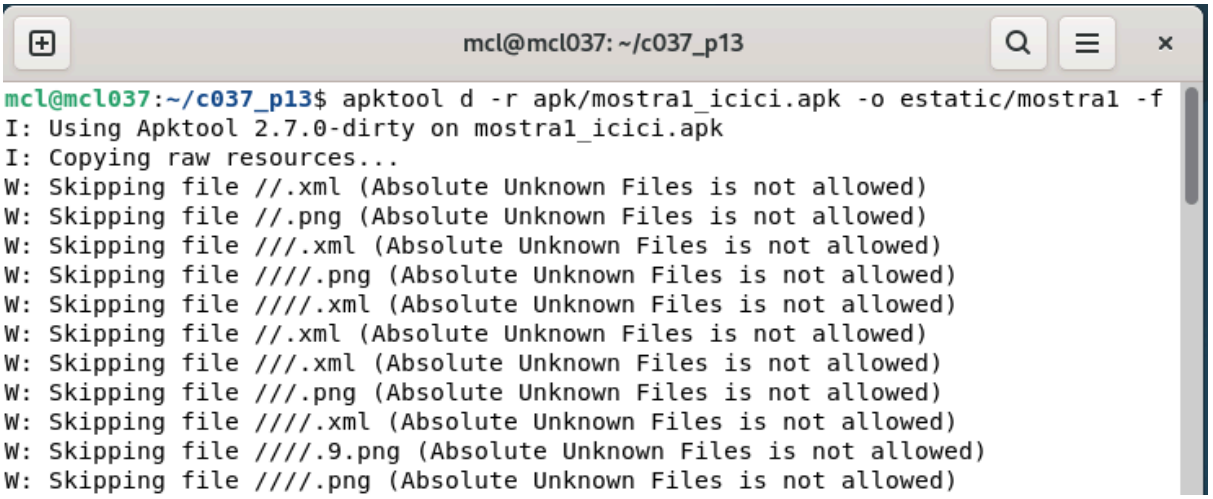
```
mcl@mcl037:~/c037_p13/eines$ apktool --version
2.7.0-dirty
mcl@mcl037:~/c037_p13/eines$ ./jadx/bin/jadx --version
1.5.1
mcl@mcl037:~/c037_p13/eines$ adb --version
Android Debug Bridge version 1.0.41
Version 34.0.5-debian
Installed as /usr/lib/android-sdk/platform-tools/adb
Running on Linux 6.12.111+deb13-amd64 (x86_64)
mcl@mcl037:~/c037_p13/eines$ ls ~/c037_p13/eines/dex-tools-v2.4/
bin                             d2j_invoke.bat
d2j-apk-sign.bat                d2j_invoke.sh
d2j-apk-sign.sh                 d2j-jar2dex.bat
d2j-asm-verify.bat              d2j-jar2dex.sh
d2j-asm-verify.sh               d2j-jar2jasmin.bat
d2j-baksmali.bat                d2j-jar2jasmin.sh
d2j-baksmali.sh                 d2j-jar-access.bat
d2j-class-version-switch.bat    d2j-jar-access.sh
d2j-class-version-switch.sh     d2j-jar-weaver.bat
d2j-decrypt-string.bat          d2j-jar-weaver.sh
d2j-decrypt-string.sh           d2j-jasmin2jar.bat
d2j-dex2jar.bat                 d2j-jasmin2jar.sh
d2j-dex2jar.sh                  d2j-smali.bat
d2j-dex2smali.bat               d2j-smali.sh
d2j-dex2smali.sh                d2j-std-apk.bat
d2j-dex-recompute-checksum.bat  d2j-std-apk.sh
d2j-dex-recompute-checksum.sh   lib
d2j-dex-weaver.bat              LICENSE.txt
d2j-dex-weaver.sh               NOTICE.txt
mcl@mcl037:~/c037_p13/eines$
```

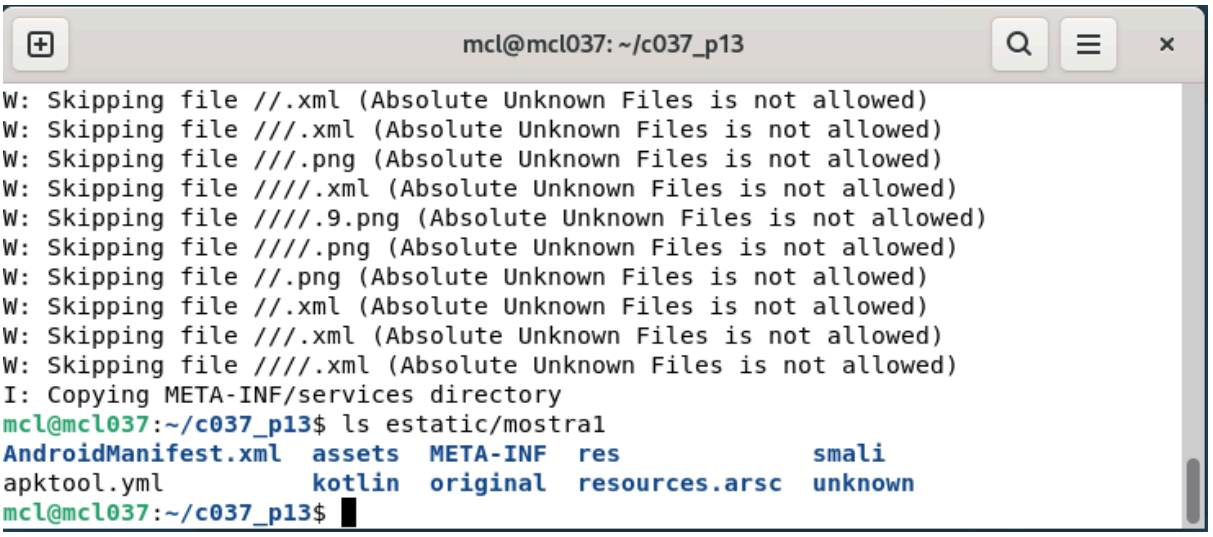
Búsqueda d'informació

1. Amb les eines proposades analitza les aplicacions instal·lades i busca la següent informació:

Permisos

He intentat descomprimir la mostra 1 amb apktool. Ha fallat en decodificar resources.arsc, així que he usat -r (sense recursos). Descomprimeix, però els avisos de fitxers amb noms buits indiquen que els recursos estan manipulats a propòsit.





Indica què fan cada un d'aquests elements a l'aplicació descarregada? (Entrega)

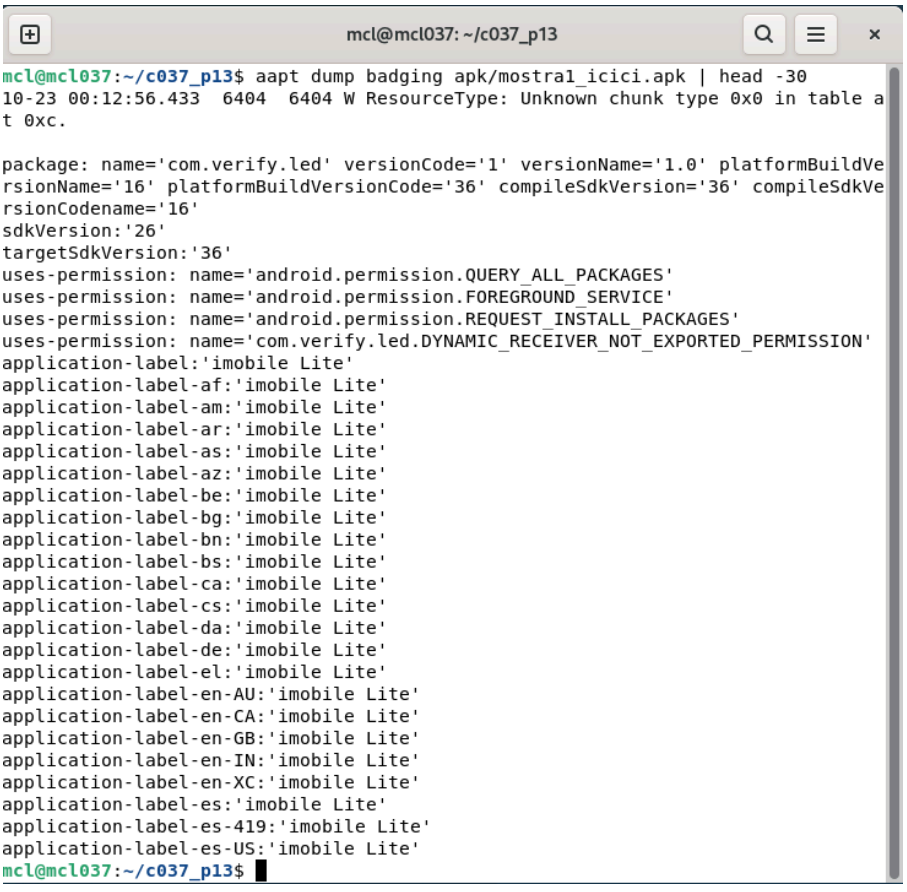
Mostra 1: ICICI (imobile Lite)

- Permisos:
    a) QUERY_ALL_PACKAGES: veu totes les apps instal·lades (per detectar apps bancàries).
    b) FOREGROUND_SERVICE: manté un servei actiu en primer pla.
    c) REQUEST_INSTALL_PACKAGES: instal·la altres APK (dropper).
    d) DYNAMIC_RECEIVER_NOT_EXPORTED_PERMISSION: permís intern per protegir receivers dinàmics.
- Activities: DynamicInstallDropSession, que demana instal·lar d'orígens desconeguts i activar la VPN (deduït del codi, perquè el manifest és il·legible).
- Services: VpnController, que rep l'ordre STOP quan acaba la instal·lació (deduït del codi).
- Receivers: InstallResultReceiver, pel nom. A més, DynamicInstallDropSession registra un receiver dinàmic per saber quan acaba la instal·lació.
- Content Providers: no n'he pogut confirmar cap (manifest il·legible). El codi importa FileProvider, així que n'hi pot haver un.

Informació de l'apk

- Nom del paquet: com.verify.led
- Tamany: 10,90 MB (11434100 bytes)
- Minimum SDK Version: 26
- Target SDK Version: 36

Com que apktool no m'ha donat el manifest, he usat aapt. D'aquí trec el paquet (com.verify.led), el Min SDK (26), el Target SDK (36) i els permisos. L'app es diu "imobile Lite", però el paquet no té res a veure amb un banc.



jadx: també falla en llegir resources.arsc, així que el manifest no es pot llegir amb ell.

```
mcl@mcl037:~/c037_p13$ ./eines/jadx/bin/jadx -d estatic/mostra1_jadx apk/mostra1
_icici.apk
INFO  - loading ...
ERROR - Failed to parse 'resources.pb'/'.arsc' file
jadx.core.utils.exceptions.JadxException: Error decode: resources.arsc
        at jadx.api.ResourcesLoader.decodeStream(ResourcesLoader.java:118)
        at jadx.core.dex.nodes.RootNode.loadResources(RootNode.java:212)
        at jadx.api.JadxDecompiler.load(JadxDecompiler.java:128)
        at jadx.cli.JadxCLI.processAndSave(JadxCLI.java:59)
        at jadx.cli.JadxCLI.execute(JadxCLI.java:37)
        at jadx.cli.JadxCLI.main(JadxCLI.java:27)
Caused by: java.io.IOException: String pool expected, expected: 0x1, actual: 0x0
, offset: 0xe
        at jadx.core.xmlgen.ParserStream.throwException(ParserStream.java:119)
        at jadx.core.xmlgen.ParserStream.checkInt16(ParserStream.java:111)
        at jadx.core.xmlgen.CommonBinaryParser.parseStringPool(CommonBinaryParse
r.java:9)
        at jadx.core.xmlgen.ResTableBinaryParser.decodeTableChunk(ResTableBinary
Parser.java:109)
        at jadx.core.xmlgen.ResTableBinaryParser.decode(ResTableBinaryParser.jav
a:84)
        at jadx.api.ResourcesLoader.decodeTable(ResourcesLoader.java:173)
        at jadx.core.dex.nodes.RootNode.lambda$loadResources$7(RootNode.java:212
)
        at jadx.api.ResourcesLoader.decodeStream(ResourcesLoader.java:113)
        ... 5 common frames omitted
INFO  - processing ...
INFO  - done
mcl@mcl037:~/c037_p13$ ls estatic/mostra1_jadx/resources
AndroidManifest.xml  play-services-basement.properties
assets               play-services-base.properties
classes.dex          play-services-tasks.properties
DebugProbesKt.bin    res
kotlin               resources.arsc
META-INF
mcl@mcl037:~/c037_p13$
```

aapt dump permissions: confirma els 4 permisos.

```
mcl@mcl037:~/c037_p13$ aapt dump permissions apk/mostra1_icici.apk
10-23 00:13:31.296  6413  6413 W ResourceType: Unknown chunk type 0x0 in table a
t 0xc.

package: com.verify.led
uses-permission: name='android.permission.QUERY_ALL_PACKAGES'
uses-permission: name='android.permission.FOREGROUND_SERVICE'
uses-permission: name='android.permission.REQUEST_INSTALL_PACKAGES'
permission: com.verify.led.DYNAMIC_RECEIVER_NOT_EXPORTED_PERMISSION
uses-permission: name='com.verify.led.DYNAMIC_RECEIVER_NOT_EXPORTED_PERMISSION'
mcl@mcl037:~/c037_p13$
```

aapt dump xmltree: retorna BAD DEPTH in XMLBlock, és a dir, el manifest és il·legible. Per això no he pogut treure els components amb eines estàndard.

```
mcl@mcl037:~/c037_p13$ aapt dump xmltree apk/mostra1_icici.apk AndroidManifest.x
ml | head -60
10-23 00:25:21.533  6476  6476 W ResourceType: Unknown chunk type 0x0 in table a
t 0xc.

***BAD DEPTH in XMLBlock: -1
mcl@mcl037:~/c037_p13$
```

Mostra 2: DMart (Dmart Shopping)

- Permisos:
  a) INTERNET: connexió a Internet (comunicació amb el servidor de l'atacant).
  b) READ_EXTERNAL_STORAGE (maxSdkVersion=28): llegeix fitxers de l'emmagatzematge.
  c) REQUEST_INSTALL_PACKAGES: instal·la altres APK (dropper/downloader).
  d) DYNAMIC_RECEIVER_NOT_EXPORTED_PERMISSION: permís intern per a receivers dinàmics.
- Activities: MainActivity, la pantalla principal que s'obre en iniciar l'app.
- Services: no n'hi ha.
- Receivers: no n'hi ha al manifest (possiblement en registra per codi).
- Content Providers: androidx.core.content.FileProvider, comparteix fitxers amb altres apps.

Informació de l'apk

- Nom del paquet: jrb.izvddlc.ufqvjeufhre
- Tamany: 2,01 MB (2110520 bytes)
- Minimum SDK Version: 21
- Target SDK Version: 33

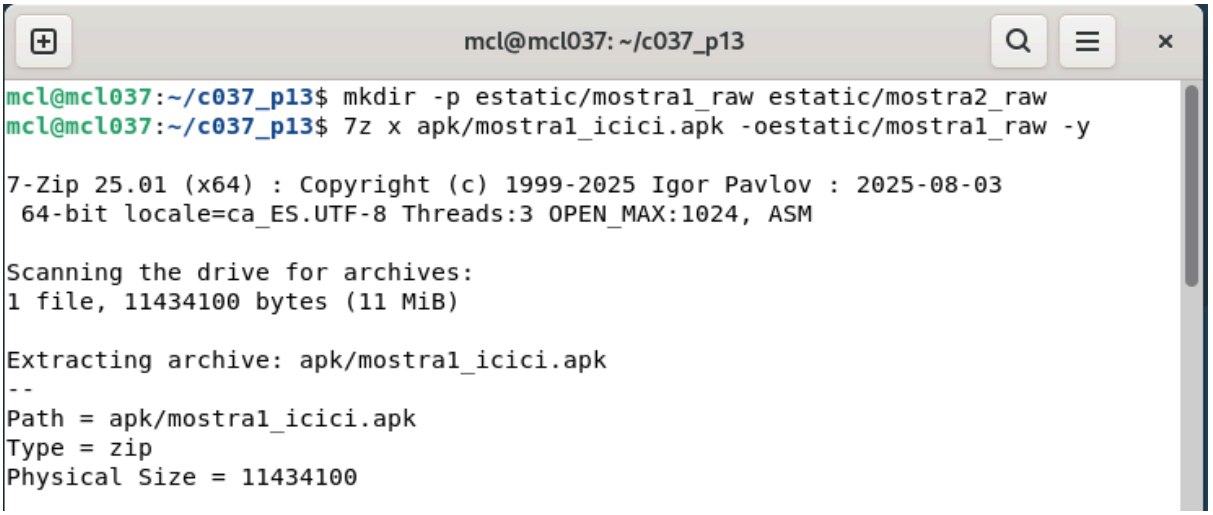
Comprovo amb aapt les dades que VirusTotal ja mostrava. Surten el paquet, els SDK (21 i 33) i 4 permisos. A l'xmltree només hi ha una activity (MainActivity) i un provider (FileProvider), sense services ni receivers.

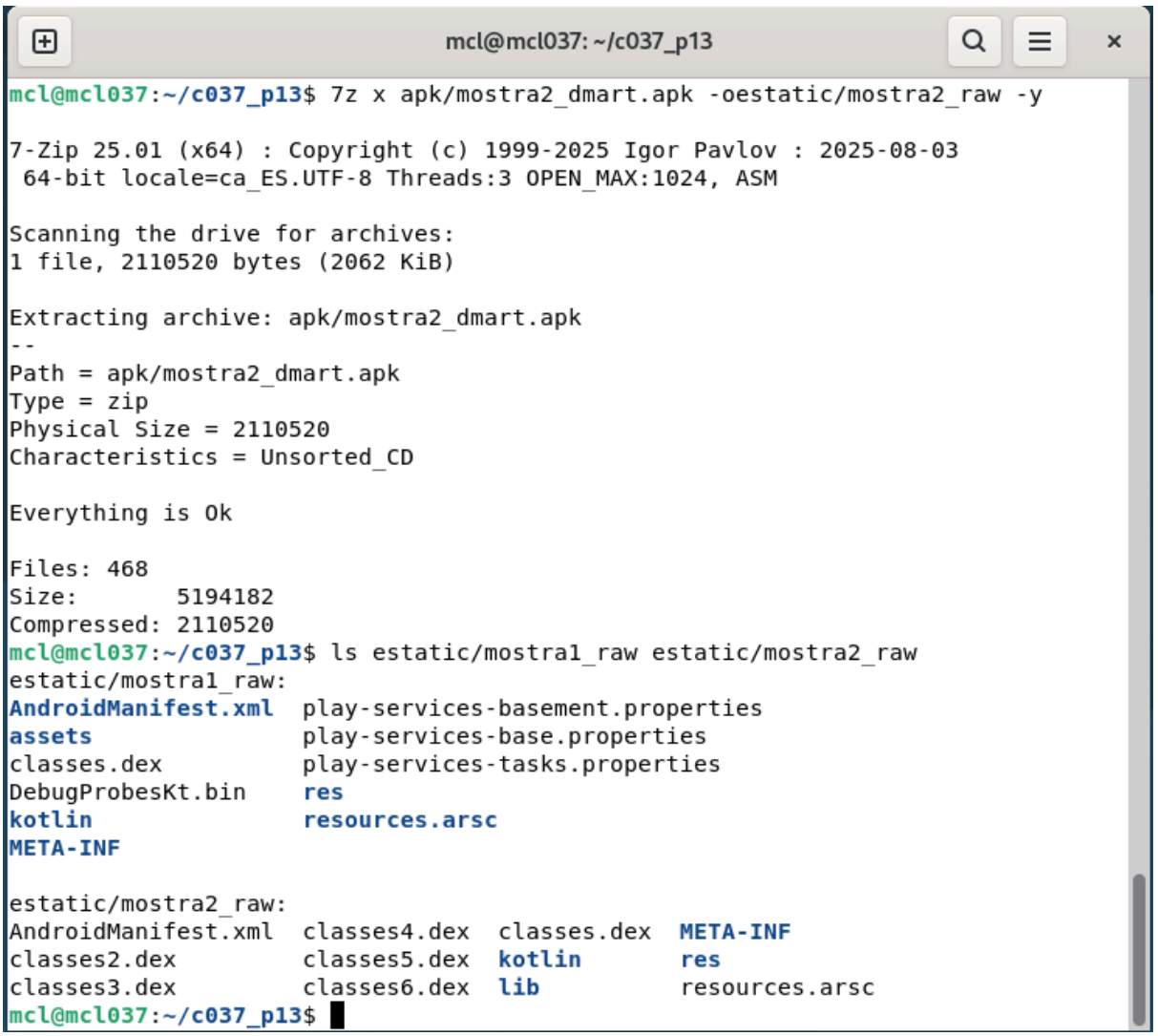
```
mcl@mcl037:~/c037_p13$ aapt dump badging apk/mostra2_dmart.apk | head -10
package: name='jrb.izvddlc.ufqvjeufhre' versionCode='5' versionName='5.0' platfo
rmBuildVersionName='13' platformBuildVersionCode='33' compileSdkVersion='33' com
pileSdkVersionCodename='13'
sdkVersion:'21'
targetSdkVersion:'33'
uses-permission: name='android.permission.INTERNET'
uses-permission: name='android.permission.READ_EXTERNAL_STORAGE' maxSdkVersion='
28'
uses-permission: name='android.permission.REQUEST_INSTALL_PACKAGES'
uses-permission: name='jrb.izvddlc.ufqvjeufhre.DYNAMIC_RECEIVER_NOT_EXPORTED_PER
MISSION'
application-label:'Dmart Shopping'
application-label-af:'Dmart Shopping'
application-label-am:'Dmart Shopping'
mcl@mcl037:~/c037_p13$ aapt dump permissions apk/mostra2_dmart.apk
package: jrb.izvddlc.ufqvjeufhre
uses-permission: name='android.permission.INTERNET'
uses-permission: name='android.permission.READ_EXTERNAL_STORAGE' maxSdkVersion='
28'
uses-permission: name='android.permission.REQUEST_INSTALL_PACKAGES'
permission: jrb.izvddlc.ufqvjeufhre.DYNAMIC_RECEIVER_NOT_EXPORTED_PERMISSION
uses-permission: name='jrb.izvddlc.ufqvjeufhre.DYNAMIC_RECEIVER_NOT_EXPORTED_PER
MISSION'
mcl@mcl037:~/c037_p13$ aapt dump xmltree apk/mostra2_dmart.apk AndroidManifest.x
ml | grep -A1 -E "E: (activity|service|receiver|provider)"
      E: activity (line=29)
        A: android:label(0x01010001)="Dmart Shopping" (Raw: "Dmart Shopping")
--
      E: provider (line=40)
        A: android:name(0x01010003)="androidx.core.content.FileProvider" (Raw: "
androidx.core.content.FileProvider")
mcl@mcl037:~/c037_p13$
```

Strings “interessants”

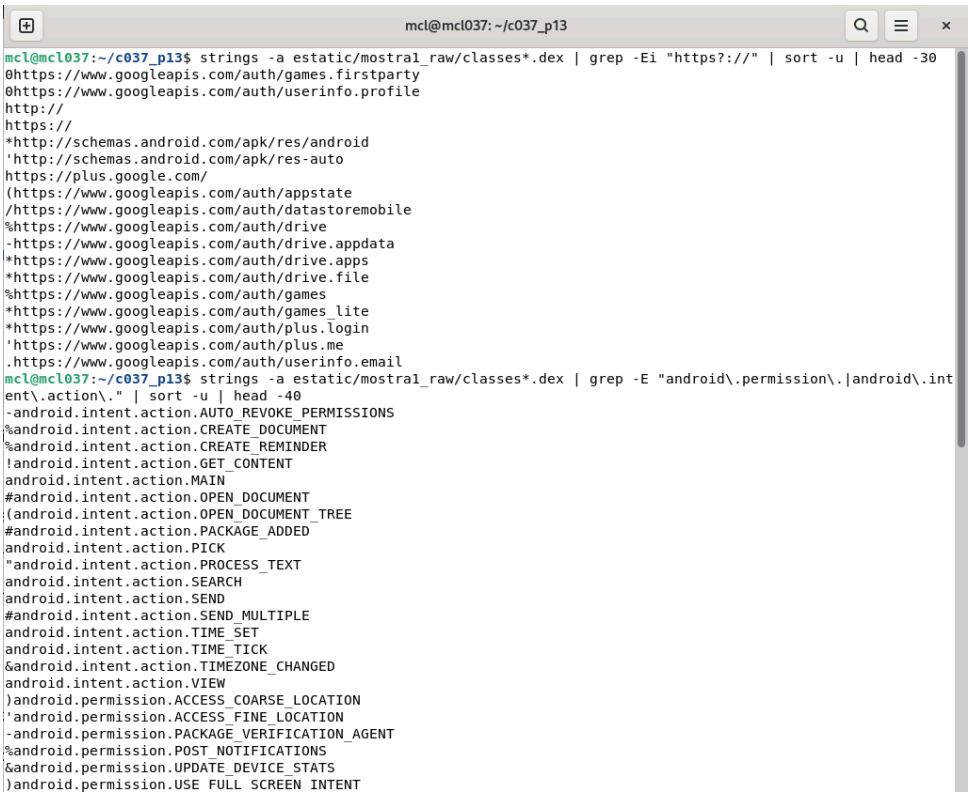
Executeu l'ordre strings de bash a diferents arxius per buscar url's, permisos a altres apps o events del sistema (encendre el mòbil, connectar les dades, etc) Feu servir filtres amb el grep. (Check)

Per buscar strings, descomprimeixo cada APK amb 7z per tenir els classes*.dex, els assets i les llibreries a mà. La mostra 1 té un sol classes.dex i la 2 en té sis.



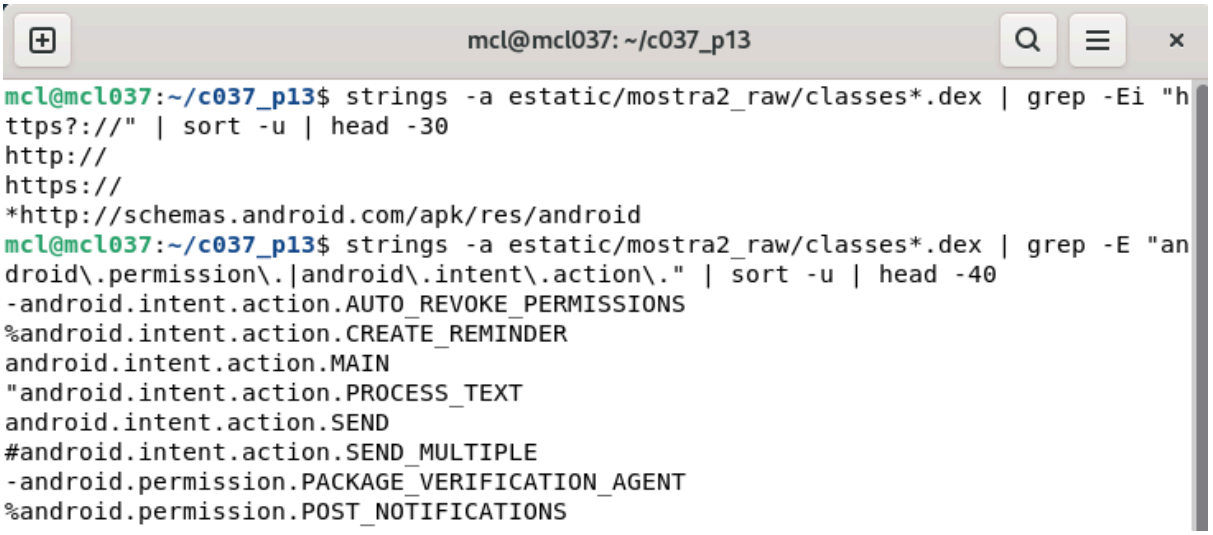


He buscat URLs, permisos i events del sistema, i paraules sospitoses als .dex. Només surten URLs de Google i classes de llibreries (AndroidX, Material). No hi ha cap servidor ni paraula clau pròpia, així que el codi maliciós no és a la vista.





He fet les mateixes tres cerques. El resultat és igual de net: només llibreries legítimes i cap URL externa. Això encaixa amb les etiquetes obfuscated i reflection de VirusTotal.



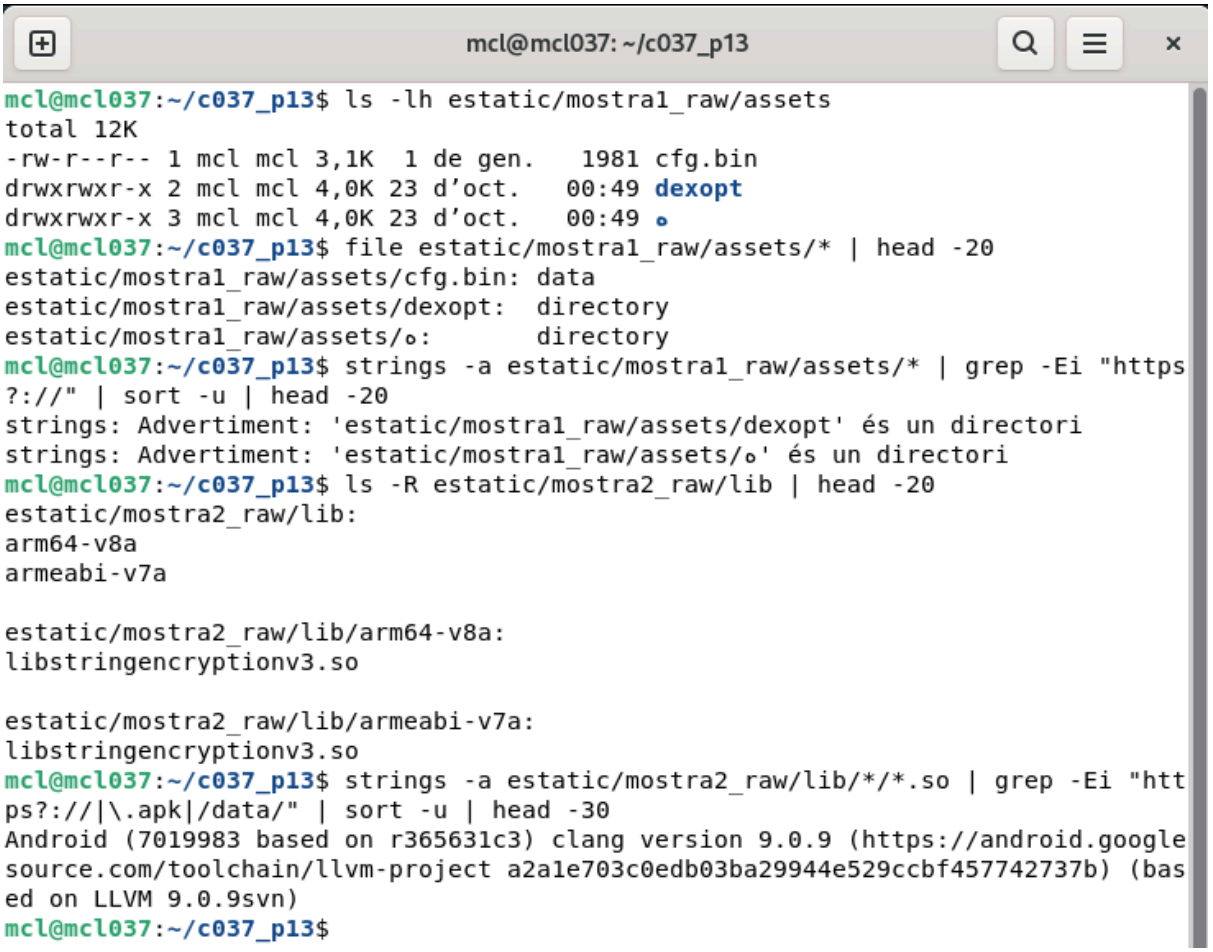


Anàlisi de l'aplicació

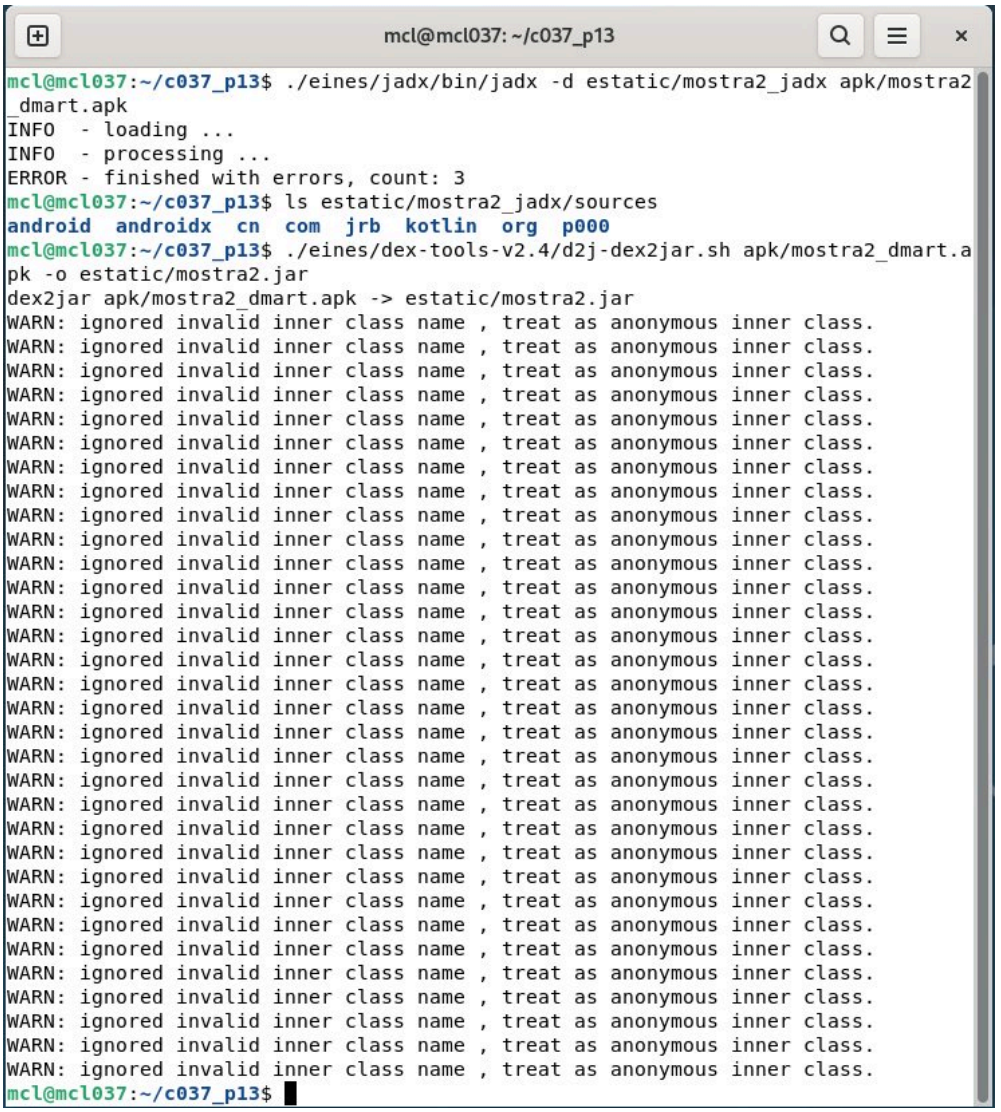
2. Descompila la aplicació amb jar2dex i/o jadx. Analitza el codi i busca quelcom que et cridi l'atenció (Check)

Ja que els .dex no revelen res, miro on pot amagar-se el codi.

a) Mostra 1: a assets hi ha cfg.bin (dades sense format conegut) i dos directoris amb noms estranys (dexopt i ◦).
b) Mostra 2: inclou libstringencryptionv3.so, una llibreria nativa d'encriptació de strings. No té sentit en una app de compres, i explica per què strings no troba cap URL.

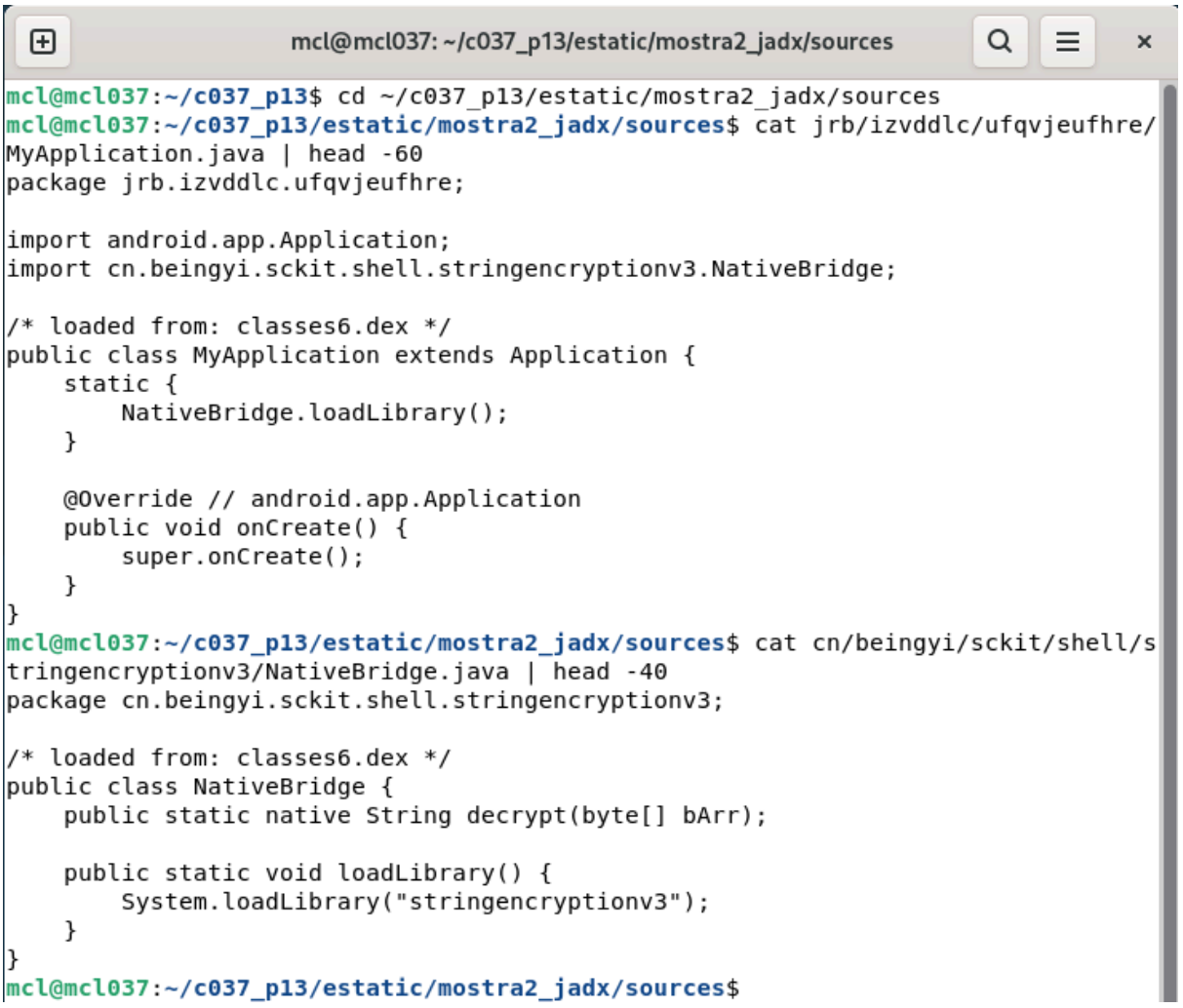


He llistat el codi propi i he buscat càrrega de llibreries natives i instal·lació d'APK. Surten MyApplication, NativeBridge i C0000, aquesta última amb un camí de paquets ofuscat (p000/p001/...).





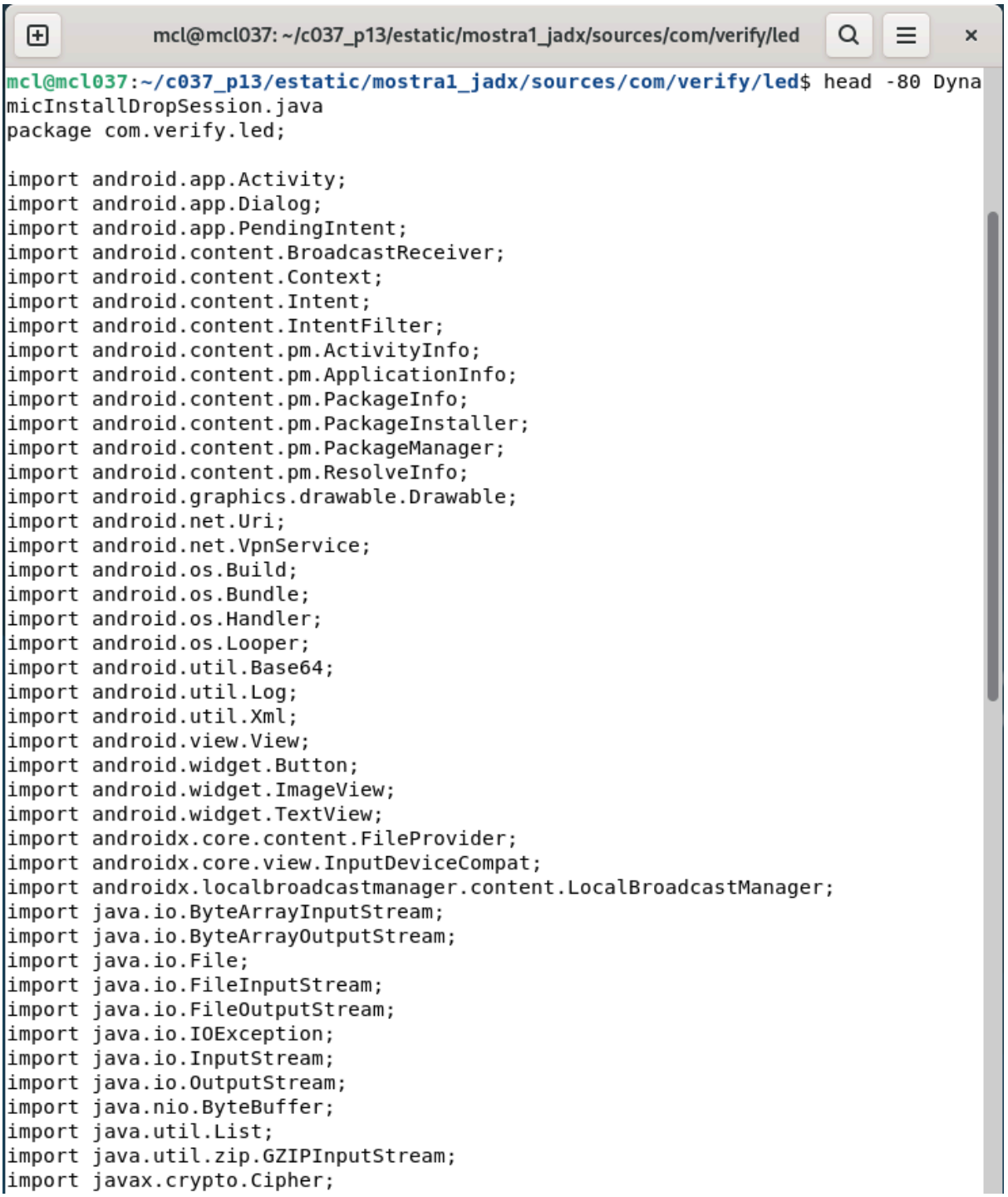
He obert les dues classes. MyApplication carrega libstringencryptionv3 en un bloc estàtic, abans de qualsevol activity, i NativeBridge té un mètode decrypt(byte[]). L'app desxifra els seus textos només en execució, i això amaga les URLs a l'anàlisi estàtic.



He buscat càrrega dinàmica i instal·lació d'APK al codi de la mostra 1. Apareix DynamicInstallDropSession.java, dins el paquet propi com.verify.led, que també conté InstallResultReceiver i VpnController.



He obert la classe. Importa Cipher, GCMParameterSpec, GZIPInputStream i Base64, i desxifra un payload a la memòria cau. També demana permís per instal·lar d'orígens desconeguts i activar una VPN, que atura quan acaba la instal·lació.



```
import javax.crypto.CipherInputStream;
import javax.crypto.spec.GCMParameterSpec;
import javax.crypto.spec.SecretKeySpec;
import org.xmlpull.v1.XmlPullParser;

/* loaded from: classes.dex */
public class DynamicInstallDropSession extends Activity {
    public static final String ACTION_INSTALL_COMPLETE = "com.verify.weird.ACTIO
N_INSTALL_COMPLETE";
    public static final String PACKAGE_INSTALLED_ACTION = "com.example.android.a
pis.content.SESSION_API_PACKAGE_INSTALLED";
    private static final int REQUEST_UNKNOWN_APP_SOURCES = 1234;
    private static final int REQUEST_VPN = 5678;
    private static final String TAG = "DynamicInstallDrop";
    private File decryptedPayloadCache;
    private Button installButton;
    private Runnable mPermissionCheckRunnable;
    private PayloadConfig payloadConfig;
    private Dialog progressDialog;
    private String targetPackageName;
    private boolean isInstallStarted = false;
    private String appLabel = "";
    private Drawable appIcon = null;
    private final Handler mHandler = new Handler(Looper.getMainLooper());
    private final BroadcastReceiver installCompleteReceiver = new BroadcastRecei
ver() { // from class: com.verify.led.DynamicInstallDropSession.1
        @Override // android.content.BroadcastReceiver
        public void onReceive(Context context, Intent intent) {
            Log.d(DynamicInstallDropSession.TAG, "installCompleteReceiver: Recei
ved broadcast.");
            try {
                context.startService(new Intent(context, (Class<?>) VpnControlle
r.class).setAction("STOP"));
            } catch (Exception unused) {
            }
            if (DynamicInstallDropSession.this.progressDialog != null && Dynamic
InstallDropSession.this.progressDialog.isShowing()) {
                Log.d(DynamicInstallDropSession.TAG, "installCompleteReceiver: D
ismissing progress dialog.");
                DynamicInstallDropSession.this.progressDialog.dismiss();
            }
            Log.d(DynamicInstallDropSession.TAG, "installCompleteReceiver: Finis
hing activity.");
            DynamicInstallDropSession.this.finish();
mcl@mcl037:~/c037_p13/estatic/mostra1_jadx/sources/com/verify/led$
```

Confirmo l'instal·lació amb grep. Es crea payload.apk a getCacheDir() i s'escriu amb PackageInstaller.createSession, openWrite i commit. És un dropper: l'app només serveix per instal·lar un segon APK.

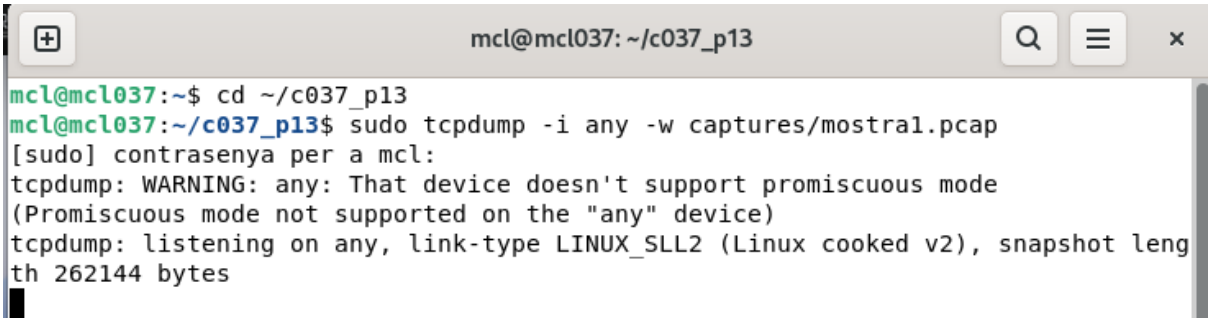
```
mcl@mcl037:~/c037_p13/estatic/mostra1_jadx/sources/com/verify/led$ grep -nE "Pac
kageInstaller|createSession|openWrite|commit|assets|\.apk" DynamicInstallDropSes
sion.java | head -30
13:import android.content.pm.PackageInstaller;
309:        File file = new File(getCacheDir(), "payload.apk");
475:            PackageInstaller packageInstaller = getPackageManager().getPacka
geInstaller();
476:            final PackageInstaller.Session openSession = packageInstaller.op
enSession(packageInstaller.createSession(new PackageInstaller.SessionParams(1)))
;
490:    static /* synthetic */ void lambda$startInstallSession$4(PackageInstalle
r.Session session, PendingIntent pendingIntent) {
492:        session.commit(pendingIntent.getIntentSender());
495:    private void addApkToInstallSession(String str, PackageInstaller.Session
 session) throws IOException {
505:                OutputStream openWrite = session.openWrite("package", 0L, -1
L);
513:                            openWrite.write(bArr, 0, read);
516:                    openWrite.flush();
517:                    session.fsync(openWrite);
518:                    if (openWrite != null) {
519:                        openWrite.close();
mcl@mcl037:~/c037_p13/estatic/mostra1_jadx/sources/com/verify/led$
```

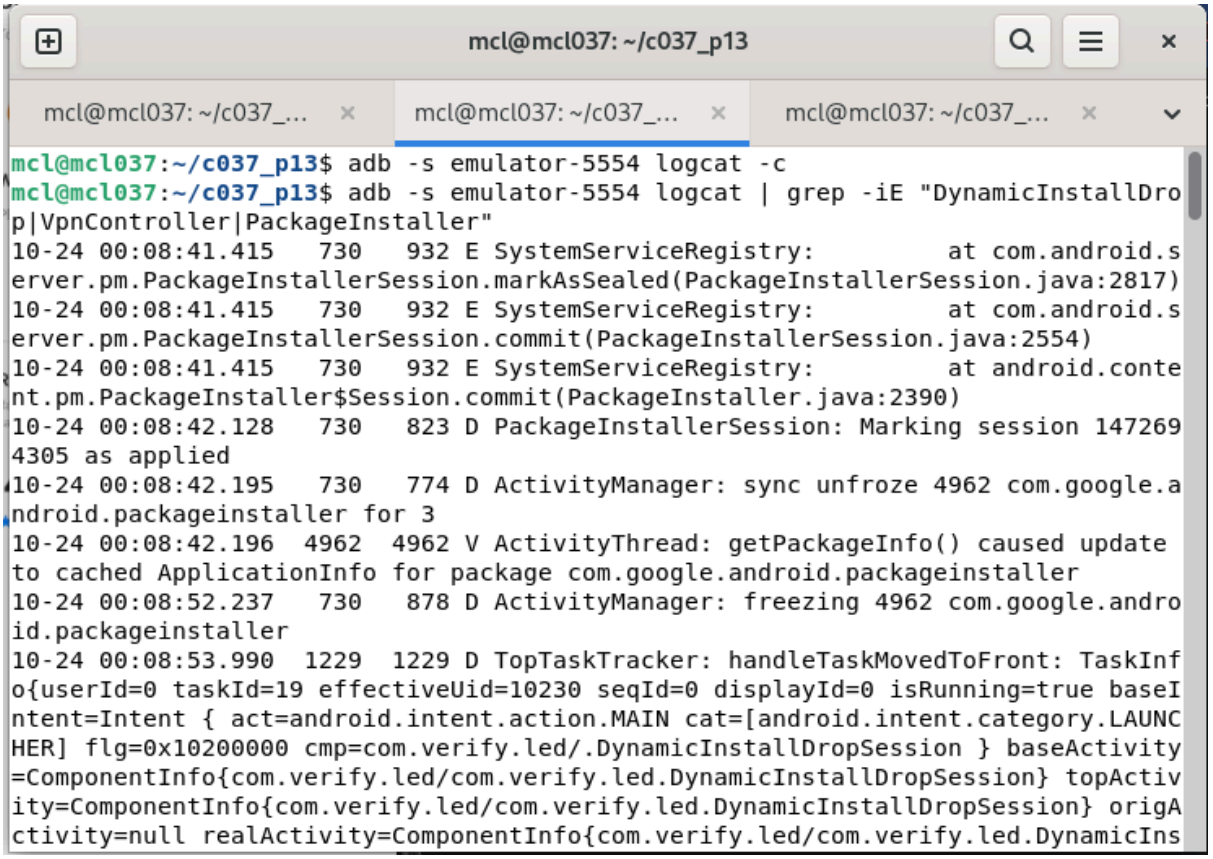
4. Anàlisi dinàmic

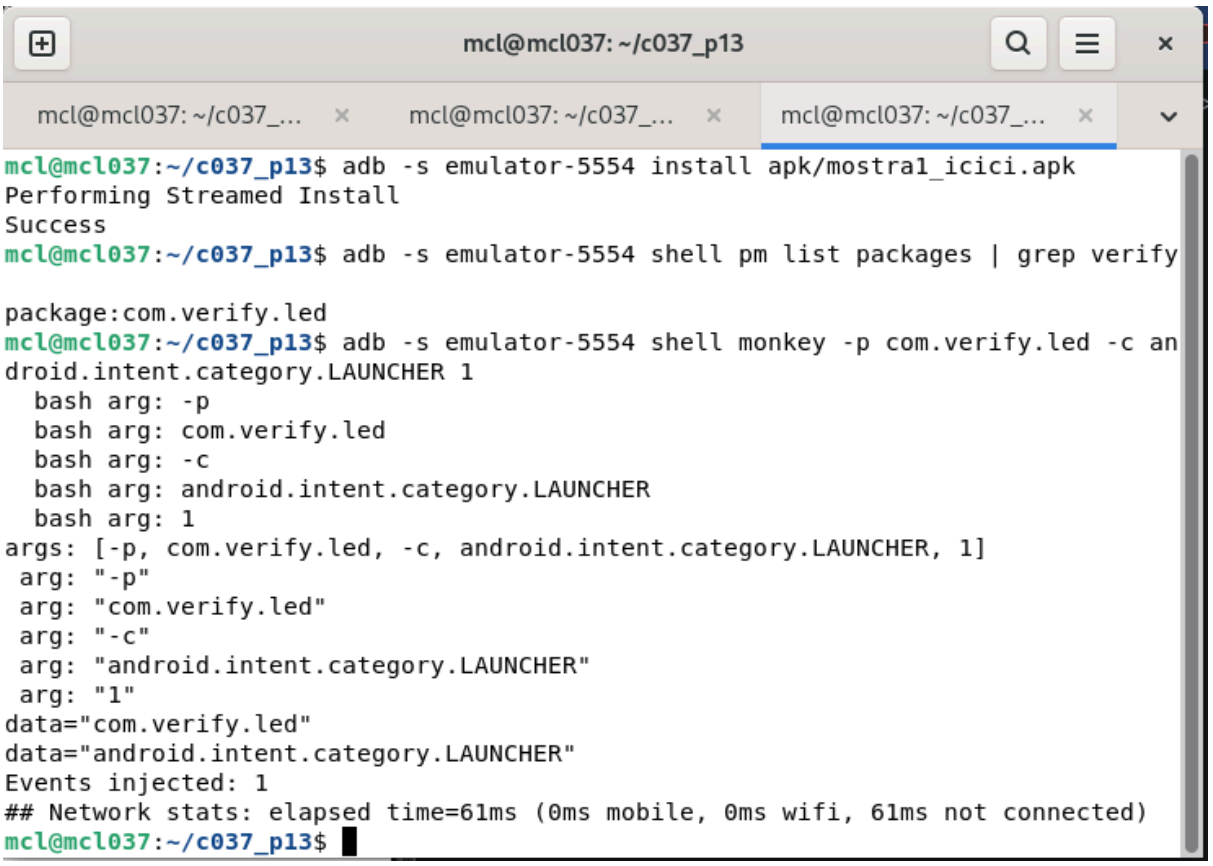
L'anàlisi dinàmic estudia l'aplicació executant-se en viu.
En aquesta part d'anàlisi, instal·larem l'aplicació a l'emulador i observarem si allò que hem trobat a l'anàlisi estàtic és coherent

Instal·lació de l'aplicació

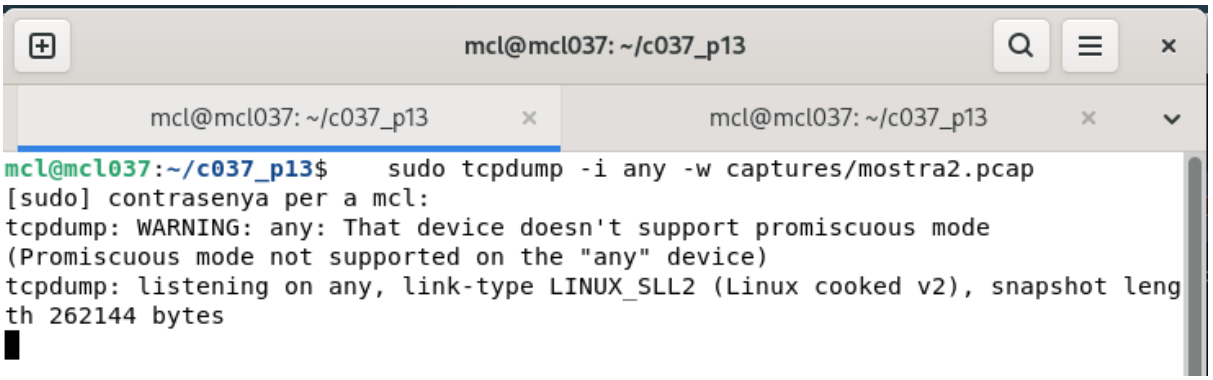
3. Instal·la les aplicacions seleccionades a un emulador y explica les ordres necessàries.(Entrega)
Abans d'instal·lar, arrenco tcpdump per guardar el tràfic a mostra1.pcap. Amb adb install instal·lo l'APK i surt Success. Amb pm list packages confirmo que és com.verify.led. Amb monkey l'obro sense conèixer l'activity, perquè el manifest és il·legible

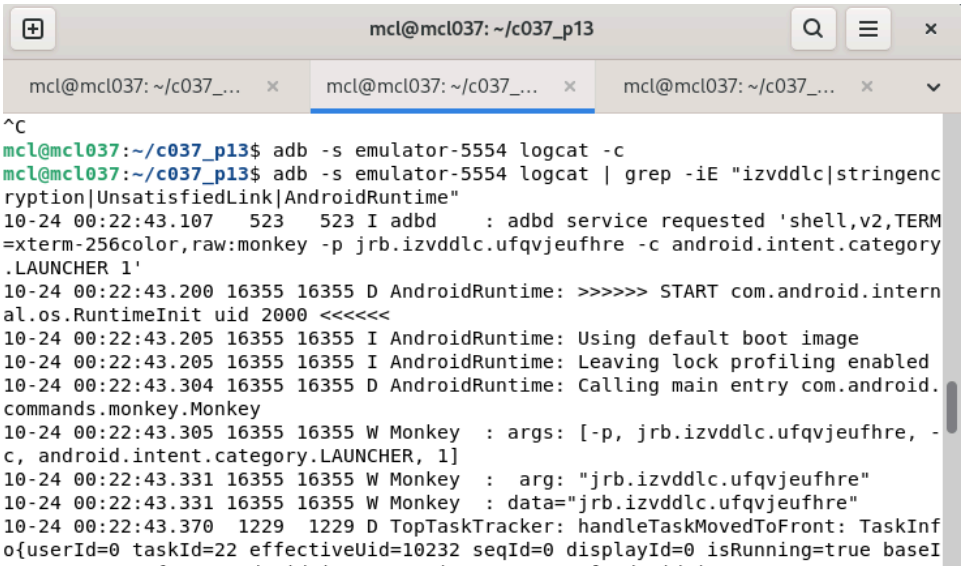


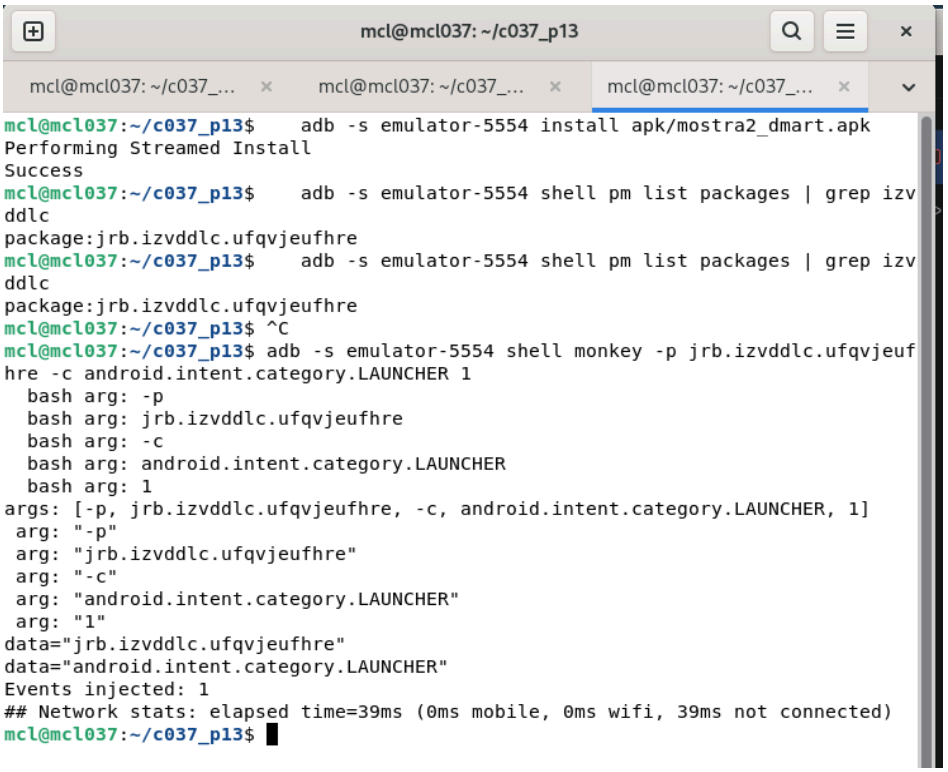



mcl@mcl037: ~/c037_p13
mcl@mcl037: ~/c037_...
mcl@mcl037: ~/c037_...
mcl@mcl037: ~/c037_...
mcl@mcl037:~/c037_p13$ adb -s emulator-5554 install apk/mostra1_icici.apk
Performing Streamed Install
Success
mcl@mcl037:~/c037_p13$ adb -s emulator-5554 shell pm list packages | grep verify
package:com.verify.led
mcl@mcl037:~/c037_p13$ adb -s emulator-5554 shell monkey -p com.verify.led -c an
droid.intent.category.LAUNCHER 1
bash arg: -p
bash arg: com.verify.led
bash arg: -c
bash arg: android.intent.category.LAUNCHER
bash arg: 1
args: [-p, com.verify.led, -c, android.intent.category.LAUNCHER, 1]
arg: "-p"
arg: "com.verify.led"
arg: "-c"
arg: "android.intent.category.LAUNCHER"
arg: "1"
data="com.verify.led"
data="android.intent.category.LAUNCHER"
Events injected: 1
Network stats: elapsed time=61ms (0ms mobile, 0ms wifi, 61ms not connected)
mcl@mcl037:~/c037_p13$

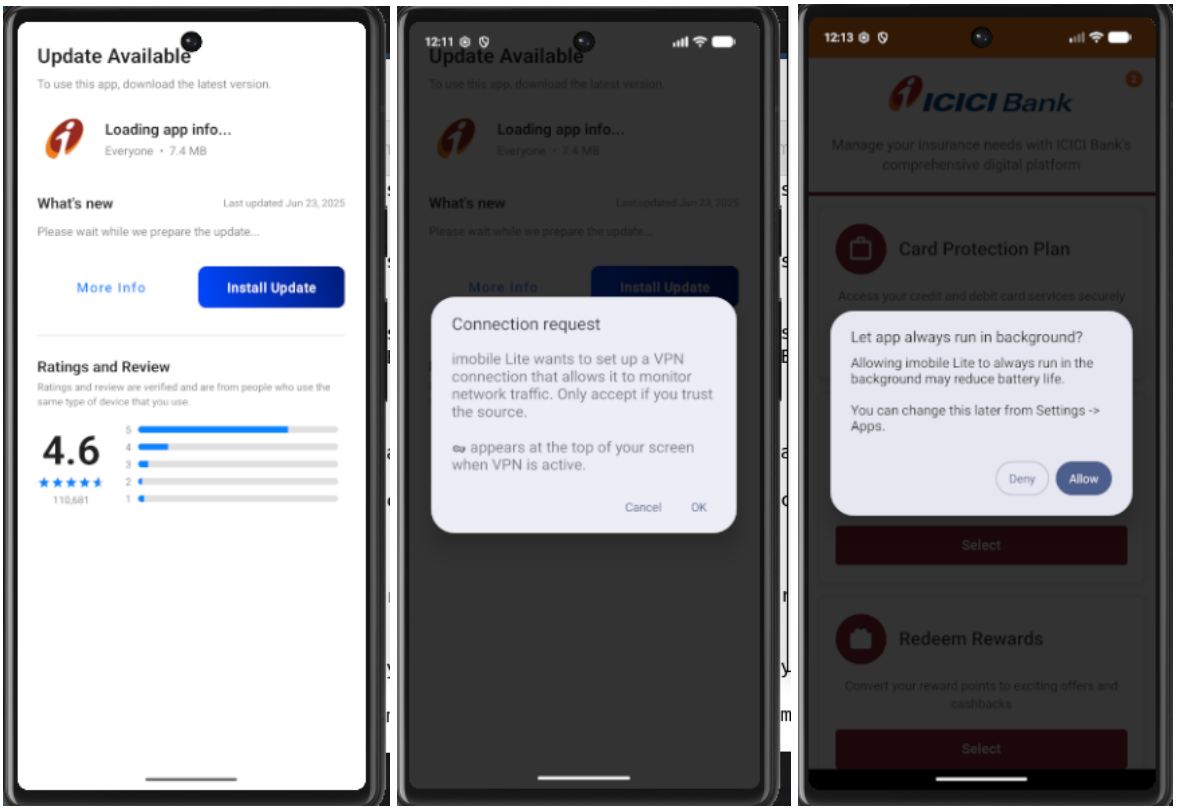
Repeteixo el procés amb la mostra 2: tcpdump cap a mostra2.pcap, logcat -c per buidar el registre i adb install amb Success. El paquet instal·lat és jrb.izvddlc.ufqvjeufhre. El logcat mostra que el sistema obre MainActivity.







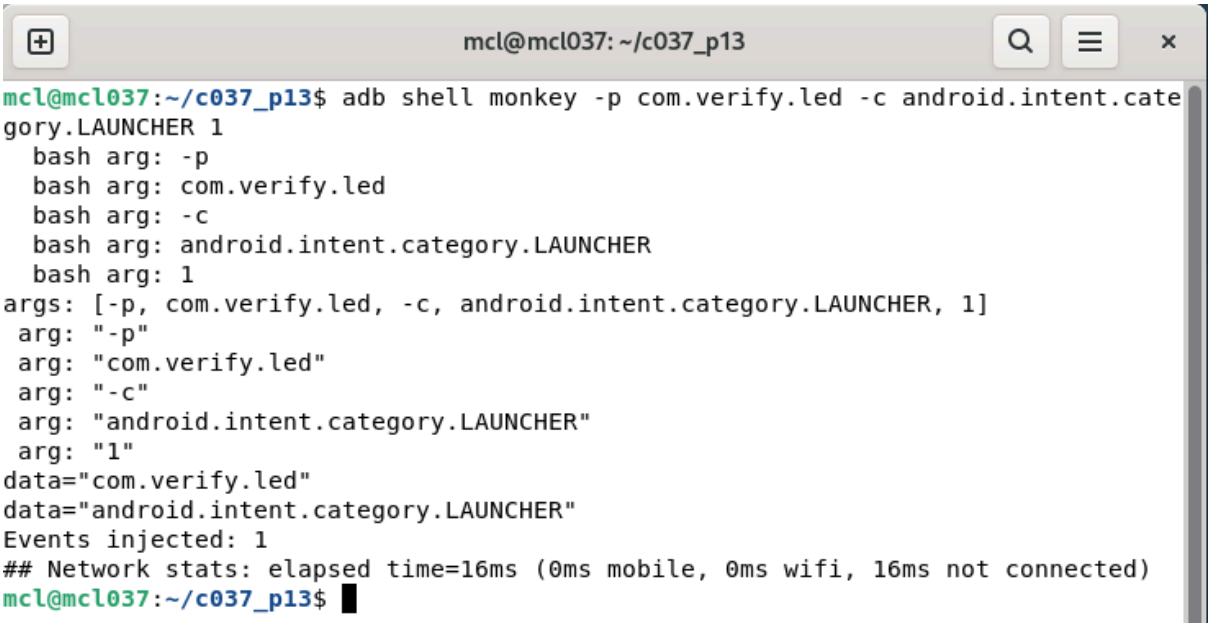
Analitza si el comportament de l'aplicació és coherent amb tot el que hem trobat en el punt 1.(Check)



A més de com.verify.led, hi ha un segon paquet, com.hdkwyo.btxdju, amb installerPackageName=com.verify.led. El va instal·lar el dropper, com vaig deduir de PackageInstaller. Demana, entre altres, READ_SMS, RECEIVE_SMS, SEND_SMS, SYSTEM_ALERT_WINDOW i RECEIVE_BOOT_COMPLETED, propis d'un troià bancari. A la primera llista també surt jrb.izvddlc.ufqvjeufhre, que és la mostra 2.

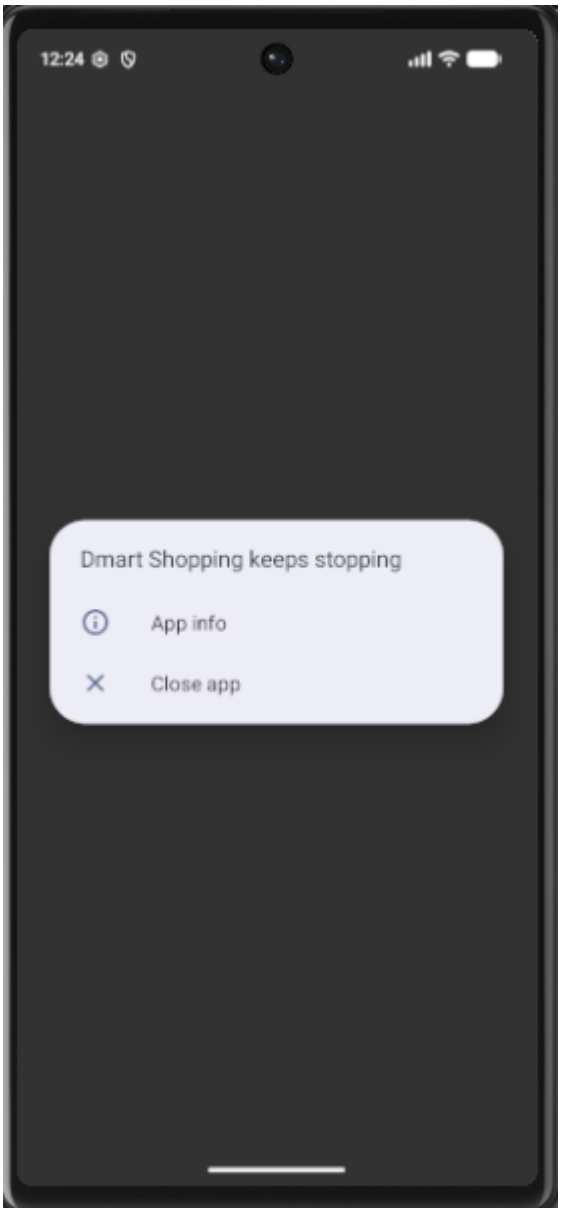
```
mcl@mcl037: ~/c037_p13
mcl@mcl037:~/c037_p13$ adb shell pm list packages -3
package:com.verify.led
package:jrb.izvddlc.ufqvjeufhre
package:com.hdkwyo.btxdju
mcl@mcl037:~/c037_p13$ adb shell dumpsys package com.hdkwyo.btxdju | grep -E "ve
rsionName|installerPackageName|android.permission" | head -30
    versionName=1.0
    installerPackageName=com.verify.led
      android.permission.READ_SMS
      android.permission.ACCESS_LOCAL_NETWORK
      android.permission.SYSTEM_ALERT_WINDOW
      android.permission.FOREGROUND_SERVICE
      android.permission.RECEIVE_BOOT_COMPLETED
      android.permission.RECEIVE_SMS
      android.permission.REQUEST_IGNORE_BATTERY_OPTIMIZATIONS
      android.permission.INTERNET
      android.permission.READ_PHONE_STATE
      android.permission.USE_FULL_SCREEN_INTENT
      android.permission.SEND_SMS
      android.permission.CALL_PHONE
      android.permission.FOREGROUND_SERVICE_DATA_SYNC
      android.permission.ACCESS_NETWORK_STATE
      android.permission.CAMERA
      android.permission.SCHEDULE_EXACT_ALARM
      android.permission.WAKE_LOCK
      android.permission.FOREGROUND_SERVICE: granted=true
      android.permission.RECEIVE_BOOT_COMPLETED: granted=true
      android.permission.REQUEST_IGNORE_BATTERY_OPTIMIZATIONS: granted=true
      android.permission.INTERNET: granted=true
      android.permission.USE_FULL_SCREEN_INTENT: granted=true
      android.permission.FOREGROUND_SERVICE_DATA_SYNC: granted=true
      android.permission.ACCESS_NETWORK_STATE: granted=true
      android.permission.WAKE_LOCK: granted=true
        android.permission.READ_SMS: granted=true, flags=[ USER_SET|USER_SENSITI
VE_WHEN_GRANTED|USER_SENSITIVE_WHEN_DENIED|RESTRICTION_INSTALLER_EXEMPT]
        android.permission.ACCESS_LOCAL_NETWORK: granted=true, flags=[ REVOKE_WH
EN_REQUESTED|USER_SENSITIVE_WHEN_GRANTED|USER_SENSITIVE_WHEN_DENIED]
        android.permission.RECEIVE_SMS: granted=true, flags=[ USER_SET|USER_SENS
ITIVE_WHEN_GRANTED|USER_SENSITIVE_WHEN_DENIED|RESTRICTION_INSTALLER_EXEMPT]
mcl@mcl037:~/c037_p13$
```

Torno a obrir l'app com.verify.led amb monkey, perquè no conec l'activity: el manifest és il·legible. Ho faig per veure si el seu comportament coincideix amb el que vaig deduir al codi.



El sistema inicia com.verify.led/.DynamicInstallDropSession i arrenca el procés de l'app. Aquesta és la classe que vaig trobar a l'estàtic, amb el payload.apk i PackageInstaller. Confirma que és l'activity principal.

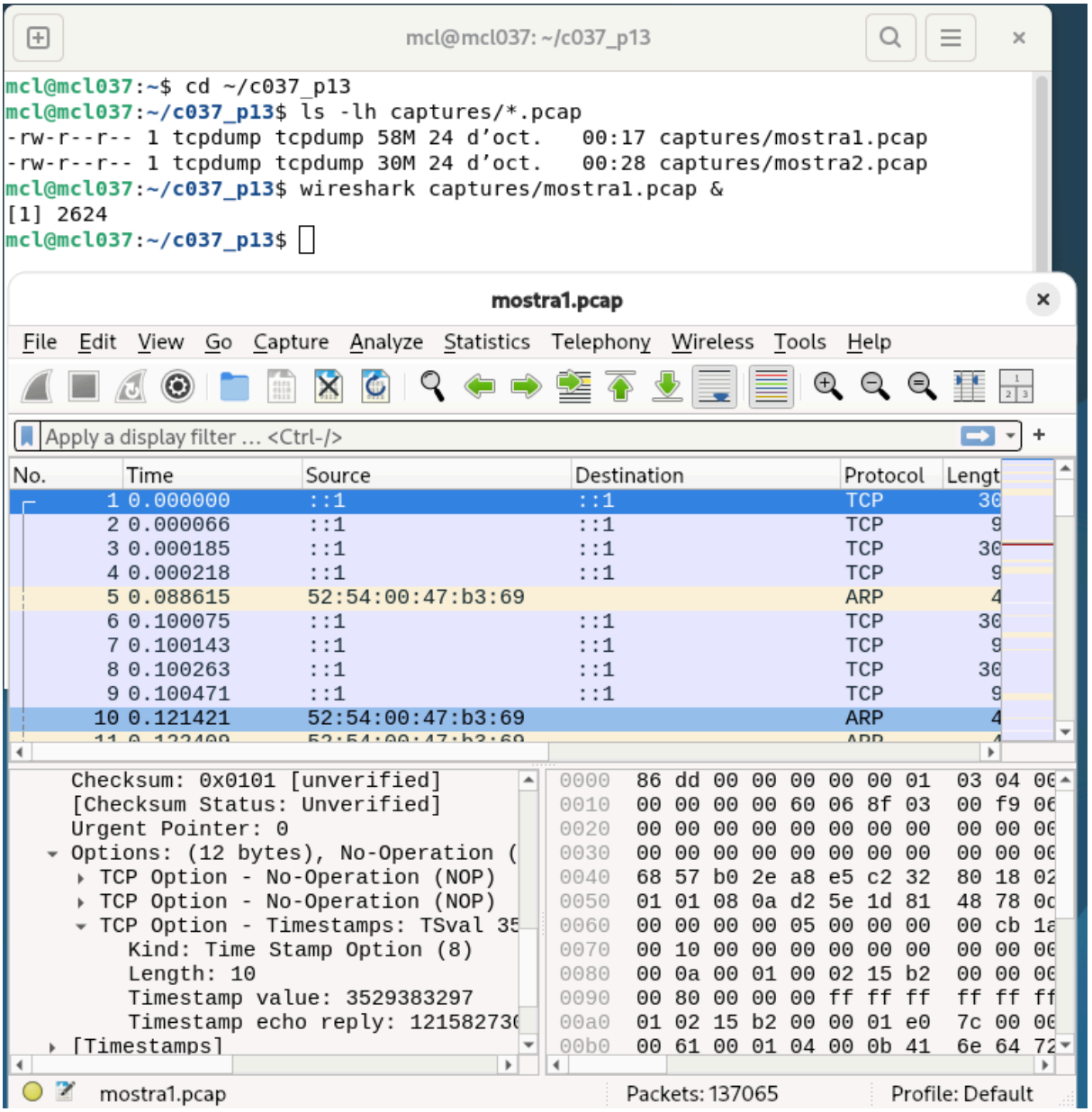




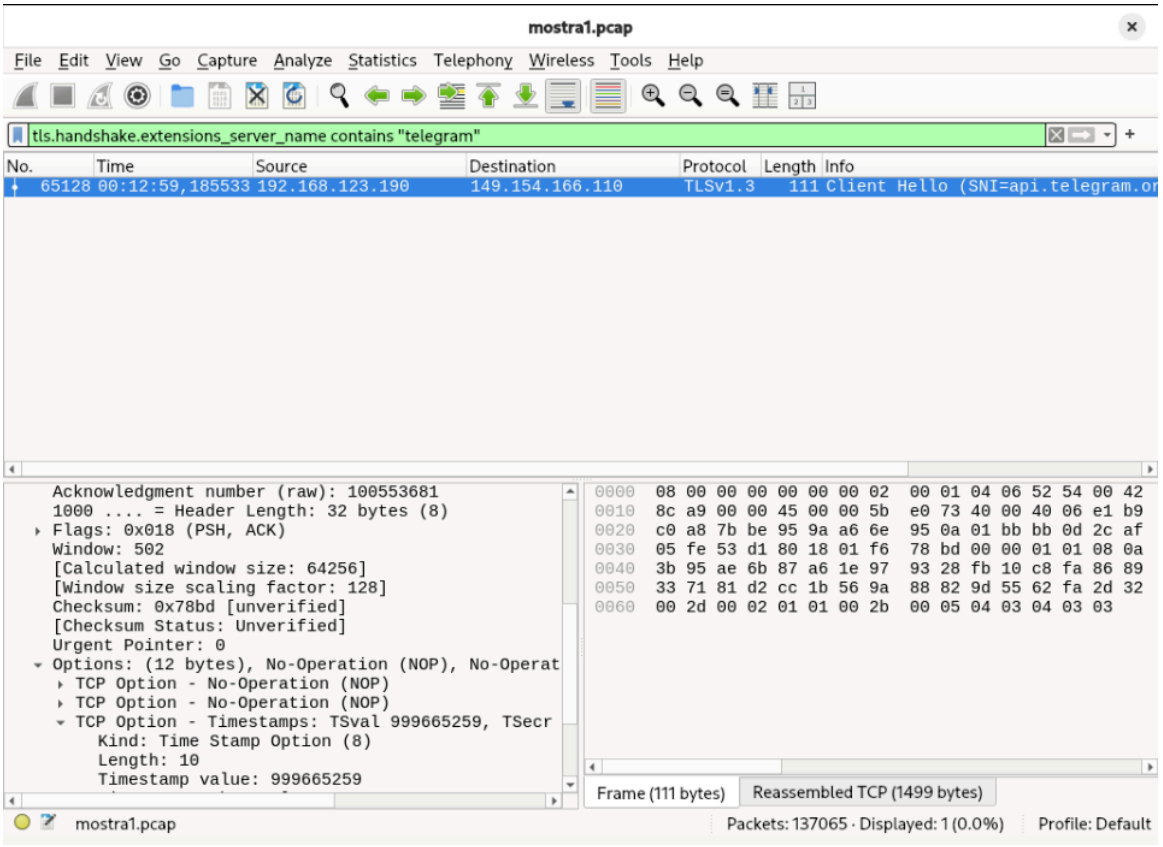
L'app s'obre amb MainActivity, la mateixa que vaig veure a VirusTotal i a jadx, i en pocs segons es tanca amb aquest missatge. No he determinat la causa al logcat. Una hipòtesi és que libstringencryptionv3.so, que només té versions ARM, no carrega en un emulador x86_64, però no ho he comprovat. Com que no s'executa, només puc confirmar la main activity.

5. Anàlisi de xarxa

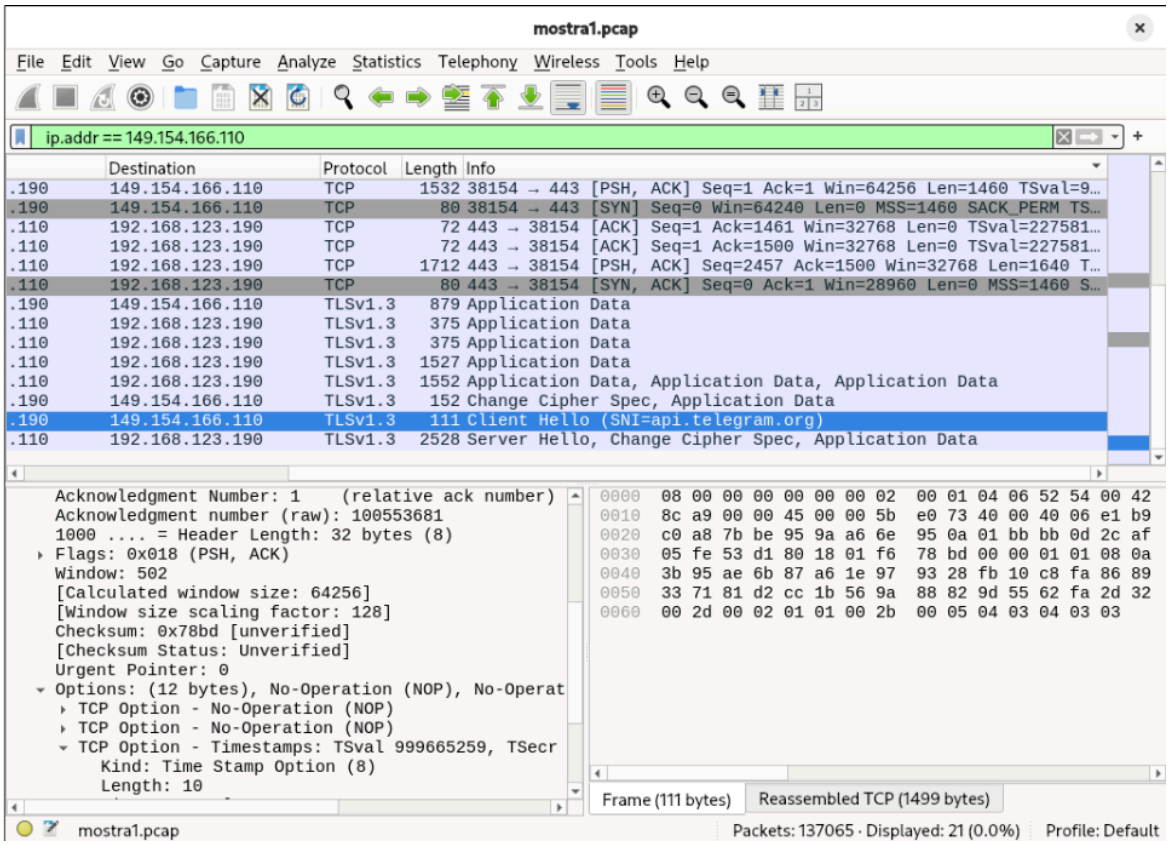
Fent servir tcpdump i/o wireshark, captura les dades que se estan enviant i/o rebent. (Entrega)
Analitza les captures, i anota qualsevol comportament anòmal.(Entrega/Check)
Pots fer servir els filtres http/s, observar si una app que no fa servir internet genera tràfic, o qualsevol detall que et cridi l'atenció.
Gairebé tots els dominis són de Google o Android (play.google.com, googleapis.com, gstatic.com) o de la VM (resources.jetbrains.com). L'única excepció és api.telegram.org.



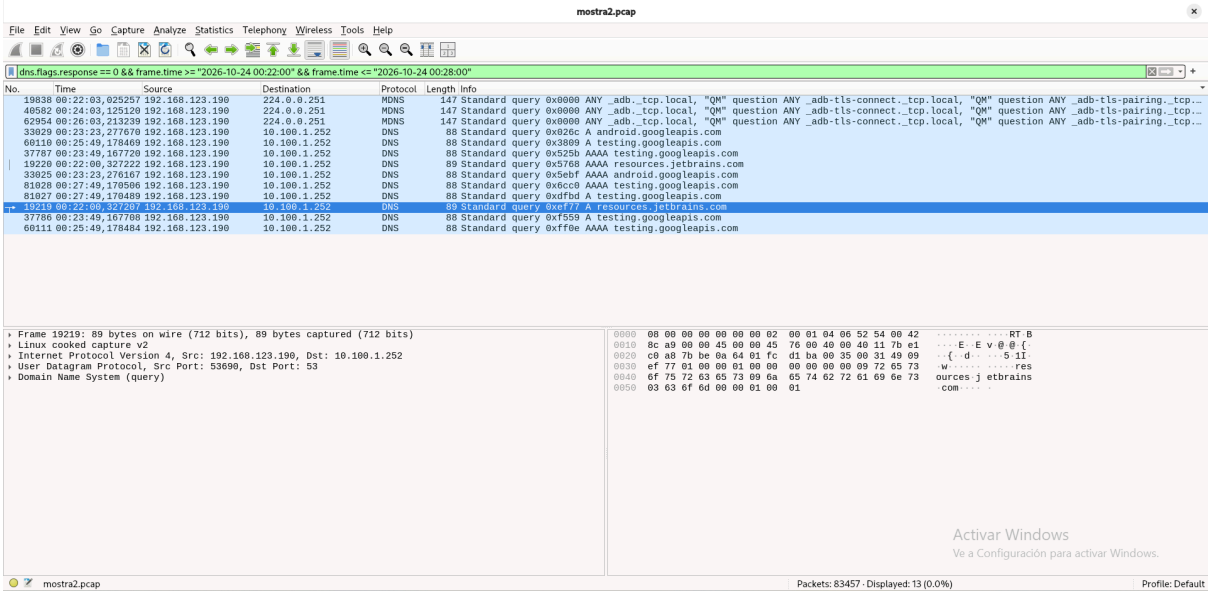
Surt un Client Hello TLS a les 00:12:59 cap a 149.154.166.110, amb SNI api.telegram.org. Aquest és el moment en què s'executa el payload.



Hi ha 21 paquets: la connexió TCP s'estableix, es negocia TLS 1.3 i s'intercanvien dades xifrades en els dos sentits.



A l'interval de la mostra 2 només hi ha dominis de Google i Android, mDNS d'adb i resources.jetbrains.com. api.telegram.org no hi apareix. No observo tràfic propi de l'app, que es tanca en pocs segons.



6. Referències

https://apktool.org/
https://github.com/pxb1988/dex2jar
https://github.com/skylot/jadx
http://java-decompiler.github.io/

Analyzers

https://malwareanalysis.co/resources/tools/android/
https://github.com/ytisf/theZoo
https://github.com/cryptwareapps/Malware-Database
https://bazaar.abuse.ch/browse/tag/android/
https://github.com/andrototal-org

*Del Llibre "Cybersecurity Ops with bash. Attack, defend, and analyze from the Command Line"